MIFA: Missing-Aware Filtered Approximate Nearest Neighbor Search over Vector-Scalar Hybrid Data

Weilong Ren
Shenzhen Institute of Computing Sciences
Shenzhen, China
renweilong@sics.ac.cn

Mengyi Yan
Shandong University
Jinan, China
yanmy@sdu.edu.cn

Yutong Ye
Beihang University
Beijing, China
yutongye@buaa.edu.cn

ABSTRACT

Filtered approximate nearest-neighbor (ANN) search retrieves embedding-near tuples satisfying scalar predicates, but existing methods assume that all predicate attributes are observed. In practice, attributes are often missing because of delayed extraction, ingestion lag, or schema evolution. Dropping incomplete tuples may exclude true answers, whereas inaccurate imputation may admit ineligible ones. We formalize *incomplete filtered ANN search* over numerical and categorical predicates and present MIFA, a missing-aware framework that jointly optimizes eligibility inference and ANN execution. MIFA makes sound hard decisions from observed values, and jointly estimates unresolved predicates using scalar and embedding-local evidence. It interleaves eligibility inference with ANN search, safely pruning noncompetitive regions and deferring statistical inference to tuples that may still affect the top-k results. It also accelerates execution through residency-preserving candidate streams that avoid relation-sized intermediate state and unnecessary data movement. We establish hard-decision soundness, a finite-sample guarantee for held-out threshold selection, exact execution over the fixed predicted-eligible set, and a $(1+\epsilon)$ distance-boundary guarantee for approximate execution. Experiments on ten real-world hybrid workloads with controlled missingness show that MIFA improves mean F_1 by 3.2–22.3 points over the best-performing baseline across three missingness mechanisms, while delivering a 1.90× P95-latency speedup and 2.02× the throughput of the fastest baseline.

PVLDB Reference Format:
Weilong Ren, Mengyi Yan, and Yutong Ye. MIFA: Missing-Aware Filtered Approximate Nearest Neighbor Search over Vector-Scalar Hybrid Data. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://anonymous.4open.science/r/MIFA_Release-68D6.

1 INTRODUCTION

Filtered or hybrid vector queries retrieve embedding-near tuples subject to structured predicates and are widely used in product search, entity-centric analytics, and retrieval-augmented applications [5, 11, 18]. Recent filtered approximate nearest-neighbor (ANN) methods interleave vector traversal with scalar filtering [3, 19, 20, 29], but typically assume that every predicate attribute is observed and its outcome is decidable.

This assumption often fails in practice. Scalar attributes may be missing or delayed because of ingestion pipelines, cold-start entities, partial extraction, or schema evolution [8, 31, 34]. A predicate referencing such an attribute evaluates to `UNKNOWN` under SQL semantics [12, 21]. Systems commonly either drop incomplete tuples or fill their missing values before executing the query. However, dropping may exclude a true nearby answer and cause a farther tuple to replace it, whereas inaccurate imputation may admit an ineligible tuple. Moreover, an eager *first-impute-then-query* pipeline reconstructs values for tuples that search may never visit and attributes not referenced by the query. Existing database-side and lazy imputation techniques [4, 22] reduce unnecessary reconstruction by placing or deferring imputation within relational query plans, but their decisions are driven by relational operators and tuple elimination rather than jointly accounting for predicate uncertainty, vector-distance ordering, and the evolving top-k boundary. Thus, they cannot determine whether an unresolved tuple is both predicate-eligible and distance-competitive, or restrict inference to only those tuples that may change the filtered ANN result.

Example 1: Consider retrieving the k products closest to the text query *"a lightweight laptop for travel"*, s.t. `price` ≤ 1000 and `rating` ≥ 4.0. A product description may be embedded and indexed before its offer metadata and review aggregates arrive from separate ingestion pipelines, leaving both predicate attributes temporarily missing. Dropping the tuple may exclude a truly qualifying laptop priced at $900 with a 4.6-star rating, whereas inaccurate imputation may admit one priced at $1,400 with a 3.2-star rating. □

To our knowledge, no prior work jointly addresses predicate inference and vector search for filtered ANN queries with incomplete scalar attributes. This gap raises a fundamental question: *how should a vector database answer filtered ANN queries when the scalar attributes used for filtering are incomplete?*

Challenges. Answering this question raises three challenges. First, predicate inference and vector search must be solved jointly rather than in isolation. Predicate inference determines which tuples satisfy the filter, while vector proximity determines which eligible tuples can enter the top-k; addressing either problem alone cannot reliably answer incomplete filtered ANN queries. Second, predicate inference must remain accurate with limited scalar evidence. Conventional statistical methods (*e.g.,* mean imputation) may produce unreliable estimates and thus incorrect tuple-eligibility decisions.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The system must therefore exploit all available observed evidence to accurately determine whether predicates involving missing attributes are satisfied. Third, execution must remain efficient. Both imputing missing attributes and searching high-dimensional vectors are computationally expensive, and imputation may dominate query processing when applied to many tuples. A practical solution must integrate missing-data handling with ANN search without sacrificing the efficiency of ANN processing.

Our proposal. We propose MIFA, a ***M**issing-aware **I**nference-guided **F**iltered **A**NN Search* framework that jointly resolves uncertain predicate eligibility and performs filtered vector search.

(1) MIFA determines eligibility without reconstructing missing values. Using observed predicate atoms, it classifies each tuple as a hard pass, hard fail, or unresolved. For unresolved tuples, it applies validation-selected, context-adaptive support templates that combine observed scalar attributes with embedding-local historical evidence to estimate joint predicate satisfaction directly.

(2) MIFA co-designs the eligibility policy with its inference and search structures. It uses held-out validation to select their configurations and then builds projected partition trees for multi-resolution inference contexts and safe distance pruning, a shared support index for evidence retrieval, and a plug-in ANN index for candidate seeding. It subsequently refits a fixed deployment policy and selects its eligibility threshold on separate calibration data while keeping the policy and indexes fixed.

(3) MIFA applies the resulting policy and structures through unified best-first execution. It uses plug-in ANN index to seed nearby candidates and the partition trees to ensure complete geometric coverage and safe distance pruning. It invokes statistical inference only for unresolved tuples that remain top-k-competitive on the distance-ordered frontier, avoiding work on noncompetitive tuples.

(4) MIFA supports append-heavy ingestion through a base–delta design. It inserts new tuples into a small delta tree and makes them immediately searchable, while using the frozen base tree to provide inference contexts. It periodically merges the delta into the base and rebuilds a compatible policy–index version under gradual distribution shift.

Contributions. We make the following contributions.

(1) Problem formulation (Section 3). We formulate *incomplete filtered ANN search*, where query-filter attributes may be missing, and define its complete-data top-k semantics, thereby separating predicate-inference error from ANN search error.

(2) The MIFA *framework* (Section 4.1). We present an end-to-end framework that couples a fixed predicate-eligibility policy with inference-aware, distance-ordered vector search. It has two phases: an *offline* phase that prepares reusable policy and structural knowledge, and an *online* phase that resolves only the query-specific uncertainty needed to produce the final top-k.

(3) Offline policy and index construction (Section 4.2). We develop a validation-driven offline procedure that constructs context-adaptive support templates, jointly selects a compatible structure–policy design, refits a fixed deployment policy, and calibrates its eligibility threshold on separate data.

(4) Online execution, acceleration, and guarantees (Sections 4.3–4.5). We develop a unified executor that combines ANN seeding, safe geometric pruning, and lazy predicate inference to process only tuples that may still enter the top-k, together with CPU/GPU techniques that reduce state materialization and data movement. We establish hard-decision soundness, a finite-sample threshold-selection guarantee, and exact and $(1+\epsilon)$-approximate search guarantees over the fixed predicted-eligible set.

(5) Experimental evaluation (Section 5). We evaluate MIFA on ten real-world hybrid workloads with controlled missingness against drop-missing, eager and query-time imputation, and filtered vector-search baselines. The results demonstrate its effectiveness in recovering the complete-data top-k, distinguish inference from search errors, and characterize its efficiency, scalability, index overhead, and maintenance under gradual distribution shift.

We review related work in Section 2 and conclude in Section 6.

2 RELATED WORK

Filtered vector search. Filtered vector search combines vector ranking with structured predicates via pre-, post-, or in-filtering [7, 28]. Representative works include ACORN [29], NaviX [35], SIEVE [19], and FAVOR [36], with specialized support for labels (UNG [3]) and numerical ranges (UNIFY, iRangeGraph [20, 43]; DSG, WoW [30, 41]). [45] gives a system evaluation of recent arts. These methods assume decidable predicates; their "dynamic" setting concerns index maintenance under inserts, not unresolved filters from missing scalar values. While they exploit vector–filter correlation for cost/recall, it does not resolve missing predicate values. MIFA instead estimates unresolved predicate conjunctions from scalar and embedding context and coordinates inference with distance-ordered search, remaining complementary to ANN optimizations.

Incomplete and probabilistic query processing. Incomplete-database semantics distinguish certain from possible answers [16], and SQL applies three-valued logic to null-containing predicates [12, 21]. Probabilistic databases attach uncertainty models to tuples/values to compute answer probabilities [9]; QPIAD rewrites queries using mined dependencies and value distributions to retrieve likely answers from incomplete autonomous databases [42]. MIFA neither enumerates completions nor assumes an explicit probabilistic relation model: validated constraints yield sound hard decisions, and held-out data selects the statistical estimator for unresolved predicates. Its vector top-k objective further requires inference to be coordinated with geometric pruning and termination.

Uncertain nearest-neighbor search. Probabilistic nearest-neighbor methods rank objects by their probability of being nearest under uncertain existence/location [2]. MIFA assumes exact embeddings and distances and treats uncertainty only in structured-predicate eligibility; admitted tuples therefore preserve distance order, and guarantees are relative to a fixed predicted-eligible set.

Missing-value estimation and lazy imputation. General-purpose methods reconstruct missing cells using chained equations [39] or mixed-type nonparametric imputation [38]. Data-cleaning systems provide probabilistic repair (HoloClean [33]), error detection (Raha [24]), or context-based correction (Baran [23]). These query-agnostic methods decouple cell repair from predicate conjunctions and vector ranking, allowing errors to compound

in eligibility decisions. ZIP instead defers imputation when downstream operators may discard a tuple [22]. MIFA materializes no repaired values: it directly estimates joint predicate satisfaction, backs off or abstains under insufficient support, and invokes inference only for top-k-competitive tuples.

3 PROBLEM DEFINITION

This section introduces hybrid data and filtered ANN queries, defines their complete-data semantic target when scalar values are missing, and formalizes incomplete filtered ANN search.

Hybrid data and filtered ANN queries. We consider a relation D with an embedding attribute $A_{\mathbf{e}} \in \mathbb{R}^d$ and numerical or categorical scalar attributes $\bar{\mathcal{A}} = \{A_1, \ldots, A_m\}$. Embeddings are always observed, whereas scalar values may be missing, denoted by $t[A_i] = \perp$. Following prior work (*e.g.,* [29]), we assume a single embedding attribute for clarity; multiple embedding attributes can be handled analogously.

A filtered ANN query is $q = (\mathbf{q}_e, k, \phi)$, where $\mathbf{q}_e$ is the query embedding, k is the requested result size, and $\phi = \bigwedge_{j=1}^{\ell} \psi_j$ is a predicate over scalar attributes. Each atom ψ_j is a numerical comparison or range, or a categorical equality or set-membership condition. Set membership captures categorical disjunctions; arbitrary Boolean formulas are outside our scope. Tuples are ordered by increasing distance to $\mathbf{q}_e$, with ties broken by a fixed unique tuple identifier:

$$t_i \preceq_q t_j \iff \big(\mathsf{dist}(\mathbf{q}_e, t_i[A_{\mathbf{e}}]),\ \mathsf{id}(t_i)\big) \leq_{\text{lex}} \big(\mathsf{dist}(\mathbf{q}_e, t_j[A_{\mathbf{e}}]),\ \mathsf{id}(t_j)\big),$$

where $\leq_{\text{lex}}$ denotes lexicographic order. The identifier may be replaced by any deterministic total-order key, such as a monotonically assigned ingestion sequence number. When all scalar attributes are observed, the exact filtered answer is

$$\mathsf{Ans}(q, D) = \mathrm{TopK}_k\big(\{t \in D \mid \phi(t[\bar{\mathcal{A}}]) = \mathsf{true}\}, \preceq_q\big),$$

where $\mathrm{TopK}_k(S, \preceq_q)$ returns the k smallest tuples in S under $\preceq_q$, or all tuples if $|S| < k$.

Incomplete-data semantics and target. With missing scalar values, predicates follow three-valued semantics: $\phi(t[\bar{\mathcal{A}}]) \in \{\mathsf{true}, \mathsf{false}, \mathsf{unknown}\}$. A conjunction evaluates to false if any atom is false, to true if all atoms are true, and to unknown otherwise.

Let D^* denote the unobserved complete relation underlying D. It has the same tuple identifiers and embeddings, agrees with D on all observed scalar values, and assigns a value to every missing scalar entry. For each $t \in D$, let t^* denote its complete counterpart in D^*. The complete-data target is

$$E^*(q, D^*) = \{t \in D \mid \phi(t^*[\bar{\mathcal{A}}]) = \mathsf{true}\},$$
$$\mathsf{Ans}^*(q, D^*) = \mathrm{TopK}_k\big(E^*(q, D^*), \preceq_q\big).$$

Although unavailable at query time, D^* defines the semantic target independently of any inference method. It is observable only in controlled evaluations or after the missing attributes become available. This exact answer serves as the semantic reference; Section 4.3 separately formalizes exact and approximate execution over the predicted-eligible set.

Incomplete filtered ANN search. Given an incomplete relation D with missing scalar attributes and a filtered ANN query $q = (\mathbf{q}_e, k, \phi)$, *incomplete filtered ANN search* returns at most k tuples intended to match the complete-data target, *i.e.,* tuples whose

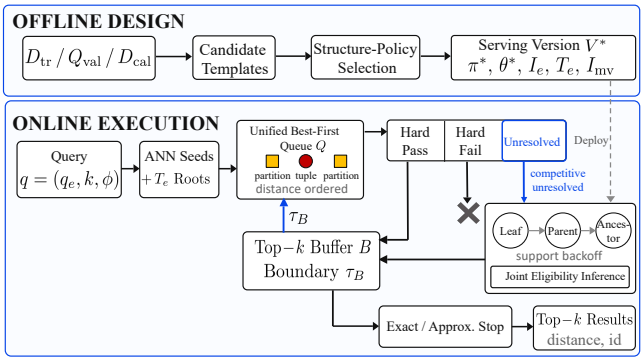


Figure 1: Architecture of MIFA.

complete counterparts satisfy ϕ and rank closest to $\mathbf{q}_e$ under $\preceq_q$. We denote the returned answer by

$$\widehat{\mathsf{Ans}}(q, D) \subseteq D, \qquad |\widehat{\mathsf{Ans}}(q, D)| \leq k.$$

We consider queries for which $\mathsf{Ans}^*(q, D^*)$ is nonempty and define $\mathrm{TP} = \big|\widehat{\mathsf{Ans}}(q, D) \cap \mathsf{Ans}^*(q, D^*)\big|$, $\mathrm{Precision} = \frac{\mathrm{TP}}{|\widehat{\mathsf{Ans}}(q,D)|}$, $\mathrm{Recall} = \frac{\mathrm{TP}}{|\mathsf{Ans}^*(q,D^*)|}$, and $F_1 = \frac{2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$. We set Precision $= 0$ when $\widehat{\mathsf{Ans}}(q, D) = \emptyset$ and $F_1 = 0$ when Precision + Recall $= 0$.

We evaluate answer quality by comparing $\widehat{\mathsf{Ans}}(q, D)$ with $\mathsf{Ans}^*(q, D^*)$ using F_1. Our objective is to produce answers with high F_1 despite missing numerical or categorical predicate attributes.

Remark. (1) We focus on read-heavy and append-oriented workloads with periodic refreshes under gradual distribution shift. Arbitrary high-churn workloads and abrupt, unrestricted shifts are outside our scope. (2) When all attributes referenced by ϕ are observed, the problem reduces to conventional filtered ANN search.

4 THE MIFA FRAMEWORK

MIFA couples a fixed predicate-eligibility policy with inference-aware, distance-ordered vector search. Section 4.1 overviews its architecture; Sections 4.2 and 4.3 present offline policy/index construction and inference-aware execution, respectively; Section 4.4 describes CPU/GPU acceleration; and Section 4.5 establishes correctness and approximation guarantees and complexity bounds.

4.1 Overview

Design overview. MIFA couples predicate inference with distance-ordered vector search. It determines eligibility directly rather than reconstructing missing values, uses validated deterministic evidence for sound hard decisions, and combines observed scalar and embedding-local evidence for unresolved tuples. It further interleaves inference with geometric search so that statistical inference is invoked only for unresolved tuples that remain top-k-competitive.

Framework. Figure 1 shows the two interacting components that implement this design.

Template-based hierarchical inference. MIFA constructs a fixed predicate-eligibility policy π offline and applies it on demand. For a tuple–predicate pair (t, ϕ), it first evaluates observed atoms and validated deterministic constraints, which may yield a sound hard

pass or fail. For an unresolved pair, it selects an applicable support template that estimates satisfaction of the residual conjunction from observed scalar attributes and embedding-local historical evidence. It starts from the tuple's leaf context and, when support is insufficient, progressively relaxes scalar matching and backs off to broader ancestor contexts. It abstains if no context provides sufficient support; otherwise, it compares the resulting score with the calibrated threshold θ, treating abstention as ineligible.

Inference-aware filtered ANN search. MIFA retrieves the distance-ordered top-k without materializing the predicted-eligible set. A plug-in ANN index provides nearby seeds, while the active partition trees ensure complete coverage and store projected MBRs that safely lower-bound the distances of their descendants. MIFA processes tree nodes and unresolved tuples in unified best-first order, prunes noncompetitive subtrees and tuples, and applies hard evaluation only to competitive candidates. It invokes statistical inference only when an unresolved tuple remains competitive at the queue head. Thus, eligibility determines admission, while distance determines ranking and when inference is needed.

Dynamic extension. MIFA extends the architecture to growing data using separate base and delta trees whose roots share the unified queue and top-k boundary. The delta tree reuses the current version's frozen projection and split tests. Within each serving period, MIFA keeps policy π, the context tree, and the support index fixed; appended tuples become immediately searchable using the base inference contexts but provide no statistical support until refresh. It periodically merges the delta into the base, rebuilds a compatible policy–structure version, and publishes the new version.

Workflow. Figure 2 summarizes MIFA's offline construction and online execution over relation states evolving through updates ΔD.

(1) Offline construction (Section 4.2). MIFA constructs candidate support templates from training data and uses held-out validation to jointly select a serving configuration $\Gamma^\star$, which specifies the index and partition design, and an eligibility policy $\pi^\star$ deciding whether a tuple is predicted eligible (Lines 1–2). It then builds the compatible partition tree, support index, and plug-in ANN index, refits $\pi^\star$, and selects on separate calibration data a threshold $\theta^\star$ that converts policy scores into eligible/ineligible decisions. MIFA publishes the resulting policy and structures as one serving version (Lines 3–7), and repeats this process after merging accumulated updates.

(2) Online execution (Section 4.3). For each query, MIFA fixes a read-consistent serving version (Line 8). It probes the ANN index for seed candidates, applies hard (non-inferential) eligibility checks, and uses hard-pass seeds to initialize the top-k buffer B, which maintains the current best eligible tuples (Line 9). It then performs unified best-first traversal with safe pruning and lazy inference for competitive unresolved tuples (Line 10). In exact mode ($\epsilon = 0$), it returns the exact top-k over the predicted-eligible set induced by $(\pi^\star, \theta^\star)$; in approximate mode ($\epsilon > 0$), it may terminate early with a $(1+\epsilon)$ distance-boundary guarantee.

Example 2: Reconsider the laptop query from Section 1, $q = (\mathbf{q}_e, 2, \text{price} \le 1000 \wedge \text{condition} = \text{new})$, where $\mathbf{q}_e$ is the query embedding. Let $\theta^\star = 0.75$ and $\epsilon = 0$. Suppose the ANN probe returns two seeds, t_1 and t_2, which pass the hard (non-inferential) eligibility checks at distances 0.18 and 0.24. MIFA uses them to initialize $B = \{t_1, t_2\}$ with top-k boundary $\tau_B = 0.24$. Unified traversal then encounters an unresolved tuple t_3, whose price and condition are missing, at distance 0.14. Because t_3 can improve B, MIFA estimates its joint predicate-satisfaction probability $\widehat{p}(t_3)$ from its observed brand, model, and hierarchical embedding context. Since $\widehat{p}(t_3) = 0.82 > \theta^\star$, it admits t_3, replaces t_2, and tightens τ_B to 0.18, allowing a remaining node with distance lower bound 0.31 to be pruned. MIFA therefore returns $\{t_3, t_1\}$ without reconstructing the missing values or inferring eligibility for pruned tuples. □

Offline input: An incomplete relation D; disjoint training, validation, and calibration relations D_{tr}, D_{val}, and D_{cal}; and labeled workloads $\mathcal{Q}_{\text{val}}$ and $\mathcal{Q}_{\text{cal}}$.
Online input: A query $q = (\mathbf{q}_e, k, \phi)$ and approximation parameter $\epsilon \ge 0$.
Output: $\widehat{\text{Ans}}_{\pi^\star,\theta^\star,\epsilon}(q, D)$.

/* Offline construction */
1. $\mathcal{T}_{\text{cand}} := \text{ConstructTemplates}(D_{\text{tr}}, \mathcal{Q}_{\text{val}})$;
2. $(\Gamma^\star, \pi^\star) := \text{SelectJointDesign}(\mathcal{T}_{\text{cand}}, D_{\text{tr}}, D_{\text{val}}, \mathcal{Q}_{\text{val}})$;
3. $(\mathcal{I}_e, \mathcal{T}_e) := \text{BuildSearchStructures}(D, A_{\mathbf{e}}; \Gamma^\star)$;
4. $\mathcal{I}_{\text{mv}} := \text{BuildSupportIndex}(D_{\text{tr}}, \mathcal{T}_e, \pi^\star)$;
5. $\pi^\star := \text{RefitDeploymentPolicy}(\pi^\star, D_{\text{tr}}, D_{\text{val}}, \mathcal{Q}_{\text{val}}, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$;
6. $\theta^\star := \text{CalibrateThreshold}(\pi^\star, D_{\text{cal}}, \mathcal{Q}_{\text{cal}}, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$;
7. $\text{PublishVersion}(D, \mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\text{mv}}, \pi^\star, \theta^\star)$;

/* Online execution */
8. $V_q := \text{PinReadVersion}()$;
9. $(B, S_{\text{reg}}) := \text{SeedHardDecisions}(\text{RetrieveCandidates}(q, V_q), q, V_q)$;
10. $B := \text{UnifiedTraverseAndInfer}(q, \epsilon, B, S_{\text{reg}}, V_q)$;
11. **return** B under $\preceq_q$;

Figure 2: The workflow of MIFA.

Remark. (1) MIFA resolves query-specific predicate eligibility without imputing missing cells or materializing the predicted-eligible set. (2) Embedding-derived contexts provide evidence for statistical inference, whereas projected MBRs serve only as safe geometric bounds for search. (3) The ANN index only seeds candidates, while tree traversal ensures complete coverage. (4) Appended delta tuples immediately join the same top-k competition but reuse frozen base contexts and contribute no statistical support until refresh.

4.2 Offline Design and Index Construction

Inputs and outputs. The offline stage takes an incomplete searchable relation D, separate training, validation, and calibration relations D_{tr}, D_{val}, and D_{cal}, and labeled workloads $\mathcal{Q}_{\text{val}}$ and $\mathcal{Q}_{\text{cal}}$. MIFA uses D_{tr} as the sole source for template discovery and statistical support; $(D_{\text{val}}, \mathcal{Q}_{\text{val}})$ to jointly select the structure configuration $\Gamma^\star$ and eligibility policy $\pi^\star$; and $(D_{\text{cal}}, \mathcal{Q}_{\text{cal}})$ only to select the eligibility threshold $\theta^\star$ after the structures and policy are fixed.

The output is a serving version comprising compatible structures $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$, a fixed predicate-eligibility policy $\pi^\star$, and a calibrated threshold $\theta^\star$. The plug-in ANN index $\mathcal{I}_e$ seeds candidates from D; the projected partition tree $\mathcal{T}_e$ supports safe distance pruning and multi-resolution inference contexts; and the shared support index $\mathcal{I}_{\text{mv}}$ retrieves observed evidence from D_{tr} across templates and contexts. For an unresolved tuple–predicate pair, $\pi^\star$ selects an applicable support template and either produces a score from the retrieved evidence or abstains. A non-abstaining score at least $\theta^\star$ yields a pass; a lower score or abstention is treated as ineligible.

We next describe the candidate support templates (Section 4.2.1), the compatible inference and search structures (Section 4.2.2), and their joint selection and finalization (Section 4.2.3).

4.2.1 *Evidence and Candidate Templates.*

Candidate support templates. For a tuple–predicate pair (t, ϕ), let Y be the attributes referenced by predicate atoms that remain unresolved after evaluating the observed values and any validated deterministic constraints. A template $r : X_s, [\mathsf{Loc}(A_\mathbf{e})] \rightsquigarrow Y$ uses training tuples matching t on an observed scalar body X_s to estimate whether the residual atoms over Y are jointly satisfied, where $X_s \cap Y = \emptyset$. The optional $\mathsf{Loc}(A_\mathbf{e})$ further restricts support to an embedding-local context; omitting it yields a scalar-only template. For the laptop query, for example, Y may contain price and condition, while X_s contains the observed brand and model.

Discovery. For each candidate target set Y, MIFA applies standard dependency-discovery methods to D_{tr}, such as conditional functional dependencies [10] and differential dependencies [37], to generate candidate bodies X_s and matching conditions. It treats these dependencies as statistical support templates rather than logically conclusive rules, so they cannot produce hard decisions. Only training tuples with all attributes in $X_s \cup Y$ observed can provide support. MIFA discards templates with insufficient support and pairs each retained scalar-only template with a location-aware variant, producing the finite candidate set $\mathcal{T}_{\text{cand}}$. See Appendix A in [1] for target/body enumeration, pseudo-missing validation instances, and cross-fitted candidate evaluation.

4.2.2 *Inference and Search Structures.*

MIFA constructs three complementary serving structures. The projected partition tree $\mathcal{T}_e$ provides safe distance bounds and multi-resolution inference contexts; the shared support index $\mathcal{I}_{\text{mv}}$ retrieves training evidence for a selected template and context; and the plug-in ANN index $\mathcal{I}_e$ supplies nearby seeds for establishing an initial top-k boundary.

Projected partition tree. Computing and storing query-to-region bounds directly in the original d-dimensional space can be expensive. MIFA therefore learns an orthogonal PCA projection P from D_{tr} and constructs $\mathcal{T}_e$ in the resulting d_p-dimensional space. Specifically, it projects an embedding $\mathbf{e} \in \mathbb{R}^d$ as

$$P(\mathbf{e}) = U_{d_p}^{\mathsf{T}}(\mathbf{e} - \boldsymbol{\mu}), \qquad U_{d_p}^{\mathsf{T}} U_{d_p} = I,$$

where $\boldsymbol{\mu}$ is the training mean and the columns of U_{d_p} are the top $d_p \ll d$ principal directions. The configuration $\Gamma = (d_p, B_{\text{leaf}})$ trades projection cost against bound tightness through d_p, while B_{leaf} controls the maximum number of training tuples per leaf.

For each candidate Γ, MIFA recursively splits the projected training tuples at the median of their largest-variance coordinate until the leaf-capacity constraint is met. After validation selects $\Gamma^\star$, MIFA fixes the resulting topology and split tests and routes the searchable relation D through them. Each leaf stores its searchable tuples, while each node N stores a projected minimum bounding rectangle $\mathsf{MBR}(N)$ enclosing its searchable descendants. Thus, D_{tr} determines the topology and inference contexts, whereas D determines the searchable contents and node bounds.

Safe node pruning. Because the orthogonal projection P is non-expansive, for Euclidean distance,

$$\mathsf{LB}(\mathbf{q}_e, N) = \min_{\mathbf{z} \in \mathsf{MBR}(N)} \|P(\mathbf{q}_e) - \mathbf{z}\|_2$$

satisfies, for every descendant tuple t,

$$\mathsf{LB}(\mathbf{q}_e, N) \le \|P(\mathbf{q}_e) - P(t[A_\mathbf{e}])\|_2 \le \mathsf{dist}(\mathbf{q}_e, t[A_\mathbf{e}]).$$

For ℓ_2-normalized embeddings, cosine distance admits the safe bound $\mathsf{LB}_{\cos}(\mathbf{q}_e, N) = \mathsf{LB}(\mathbf{q}_e, N)^2/2$. Thus, $\mathsf{LB}(\mathbf{q}_e, N)$ lower-bounds every descendant distance, allowing MIFA to prune N when this bound exceeds the current top-k boundary τ_B.

Hierarchical inference contexts. MIFA routes both target and training tuples through the same fixed split tests. For a target tuple t, the nodes on its leaf-to-root path define embedding contexts from local to progressively broader regions. Within each context, the routed training tuples provide the historical population from which $\mathcal{I}_{\text{mv}}$ retrieves matching evidence. When leaf support is insufficient, the policy backs off to broader ancestors.

Shared support index. Given a tuple t, a template $r : X_s, [\mathsf{Loc}(A_\mathbf{e})] \rightsquigarrow Y$, and a context node N, $\mathcal{I}_{\text{mv}}$ retrieves training tuples that (i) match $t[X_s]$, (ii) have observed values for Y, and (iii) when locality is required, fall within N's embedding context. These tuples provide the statistical support for jointly inferring the residual predicate on Y. See Appendix E in [1] for the shared index layout and retrieval procedure.

Plug-in ANN index. MIFA builds $\mathcal{I}_e$ over the searchable embeddings using an existing ANN method (*e.g.,* HNSW [25]). For each retrieved seed, it performs only hard (non-inferential) eligibility checks: a hard pass updates the top-k buffer, a hard fail is discarded, and an unresolved seed is deferred to coverage-preserving tree traversal. Since tree traversal, not $\mathcal{I}_e$, guarantees complete coverage, ANN recall affects efficiency but not exact-mode correctness.

4.2.3 *Structure–Policy Selection and Finalization.*

The candidate templates, tree configurations, and predeclared policy choices define a finite set C of compatible structure–policy designs. A candidate policy π specifies an ordered subset of templates, prioritizing applicable templates with richer observed scalar bodies, together with their applicability rules, minimum-support requirements, hierarchical backoff sequences, and template-specific shrinkage strengths κ_r. Offline validation selects this policy design, and the resulting template order is fixed in $\pi^\star$.

For an unresolved pair (t, ϕ), π selects the first applicable template in its fixed order, using a body-free template as fallback. For a location-aware template, it retrieves matching training evidence from t's leaf and backs off through the fixed ancestor sequence. It abstains if no applicable template/context provide sufficient support.

Hierarchical joint estimation. Suppose π selects template r, and let $N_j, \ldots, N_J$ be its backoff path from the finest sufficiently supported context N_j to the broadest allowed context N_J. Let C_i contain the training tuples within N_i that match the template body and have observed target attributes. Define $n_i = |C_i|$ and let $n_i^+(\phi)$ count the tuples in C_i satisfying the entire unresolved conjunction under the current query constants. MIFA computes

$$p_{C_J} = \frac{n_J^+(\phi)}{n_J}, p_{C_i} = \frac{n_i^+(\phi) + \kappa_r p_{C_{i+1}}}{n_i + \kappa_r} (j \le i < J), z_r(t, \phi) = p_{C_j}.$$

Here, $\kappa_r \geq 0$ controls how strongly each finer estimate is shrunk toward its broader parent estimate, which acts as κ_r pseudo-observations. Because $n_i^+(\phi)$ evaluates the entire unresolved conjunction, $z_r(t, \phi)$ estimates joint satisfaction while preserving dependencies among the unresolved predicate atoms.

Joint selection. MIFA first discards candidate pairs that fail the predeclared minimum coverage requirements across predicate–missingness cases. For each remaining pair (Γ, π), it computes $\mathsf{InfScore}(\Gamma, \pi)$, where a higher value indicates better cross-fitted joint-inference quality, and $\mathsf{TimeCost}(\Gamma, \pi)$, where a lower value indicates lower normalized cost when replaying the online executor on the validation workload. MIFA selects

$$(\Gamma^\star, \pi^\star) = \arg\max_{(\Gamma,\pi)\in C} \left[\mathsf{InfScore}(\Gamma, \pi) - \lambda\, \mathsf{TimeCost}(\Gamma, \pi)\right],$$

where $\lambda \geq 0$ controls the trade-off: larger values prioritize online efficiency. See Appendix A in [1] for the candidate space, cross-fitted scoring, replay-cost model, and final refitting procedure.

Finalization and calibration. After joint selection fixes $\Gamma^\star$ and the design of $\pi^\star$, MIFA builds the persistent structures $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\mathrm{mv}})$ and refits the policy's scoring functions using all statistically resolved validation instances. Validation labels are used for refitting, but all statistical support remains restricted to D_{tr}.

After $\mathcal{J}$ and $\pi^\star$ are fixed, MIFA selects the operating threshold from a finite predeclared set Θ by minimizing a predeclared query-level loss L (*e.g.,* $1 - F_1@k$) on the separate calibration workload:

$$\theta^\star = \arg\min_{\theta\in\Theta} \frac{1}{|Q_{\mathrm{cal}}|} \sum_{q_i \in Q_{\mathrm{cal}}} L(q_i; \pi^\star, \theta, D_{\mathrm{cal}}).$$

Calibration changes only θ: pre-mask calibration values are used solely to determine ground truth and loss, never as statistical support or tuple features. Finally, MIFA publishes the searchable snapshot D, $\mathcal{J}$, $\pi^\star$, and $\theta^\star$ as one read-consistent serving version.

4.3 Inference-Aware Filtered ANN Execution

Using the fixed deployment pair $(\pi^\star, \theta^\star)$ and compatible serving structures $\mathcal{J}$ from Section 4.2, MIFA answers q without materializing the predicted-eligible set $\widehat{E}(q, D)$. Figure 3 summarizes its execution over a fixed relation. Section 4.3.1 defines MIFA's on-demand eligibility evaluation, while Section 4.3.2 shows how MIFA interleaves it with ANN seeding, safe geometric pruning, and deferred inference and extends the executor to base–delta updates.

4.3.1 *On-Demand Policy Application.*

Hard evaluation and residual predicate (lines 3 and 18). Given a tuple–predicate pair (t, ϕ), MIFA first evaluates every atom whose referenced values are observed. A false observed atom yields a hard fail; if all atoms are observed and true, t receives a hard pass. For the remaining atoms, MIFA applies the fixed set Σ_{det} of independently supplied and validated deterministic constraints (*e.g.,* zip code determines city name). Each applicable constraint provides a certified domain guaranteed to contain the true hidden value, and MIFA intersects these domains for each missing attribute. It certifies an atom as true only if each value in the resulting domain satisfies the atom, and as false only if the domain is disjoint from its satisfying region. Any certified-false atom yields a hard fail. Let $\phi_{\mathrm{res}}(t)$ be the conjunction of atoms that remain unresolved after checking. An empty $\phi_{\mathrm{res}}(t)$ yields a hard pass; otherwise, statistical evaluation is required.

Input: A query $q = (\mathbf{q}_e, k, \phi)$, a relation D, the deployed $(\pi^\star, \theta^\star)$, its compatible structures $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\mathrm{mv}})$, and $\epsilon \geq 0$.
Output: A buffer B of at most k predicted-eligible tuples in D.

```
   /* Initialize the execution state */
1.  B := ∅; S_reg := ∅;
   /* ANN seeding: finalize only hard decisions */
2.  for each t ∈ RetrieveCandidates(I_e, q_e) do
3.    h := HardEvaluate(t, φ, π*);
4.    if h = pass then B := UpdateBuffer(B, t, q);
5.    if h ∈ {pass, fail} then S_reg := S_reg ∪ {t};
   /* Unified best-first traversal */
6.  Q := InitNodeQueue(T_e, q_e);
7.  while Q ≠ ∅ do
8.    if CanStop(Q, B, k, ε) then break;
9.    x := PopMin(Q);
10.   if x is an internal node then
11.     for each child N' of x do
12.       if |B| < k or LB(q_e, N') ≤ τ_B then
13.         Push(Q, N', LB(q_e, N'));
14.   elseif x is a leaf then
15.     for each t ∈ Tuples(x) \ S_reg do
16.       d_t := dist(q_e, t[A_e]);
17.       if CanAffect(t, B, q) then
18.         h := HardEvaluate(t, φ, π*);
19.         if h = pass then B := UpdateBuffer(B, t, q);
20.         elseif h = unresolved then Push(Q, t, d_t);
21.       S_reg := S_reg ∪ {t};
22.   else  /* x is an unresolved tuple */
23.     if CanAffect(x, B, q) then
24.       ê(x, φ) := InferEligibility(x, φ, I_mv, π*, θ*);
25.       if ê(x, φ) = 1 then B := UpdateBuffer(B, x, q);
26. return B ordered by ≤_q;
```

Figure 3: Inference-aware filtered ANN execution in MIFA.

Statistical evaluation (lines 24–25). Let $Y_\phi(t)$ be the missing attributes referenced by $\phi_{\mathrm{res}}(t)$. From the templates retained in $\pi^\star$, MIFA selects the first one whose target covers $Y_\phi(t)$ and whose body X_s is fully observed on t. If none applies, it uses the first applicable body-free template ($X_s = \emptyset$); if no such fallback exists, it abstains.

For the selected template r, MIFA uses $\mathcal{I}_{\mathrm{mv}}$ to retrieve training tuples that match $t[X_s]$ and have observed target attributes. For a location-aware template, it follows the fixed backoff sequence from t's leaf to broader ancestors until $s_{\min}(r)$ is met. It abstains if no permitted context provides sufficient support; otherwise, it computes $z_r(t, \phi)$ using the estimator in Section 4.2.3. A hard pass is eligible and a hard fail is ineligible; a statistical case is predicted eligible iff it does not abstain and $z_r(t, \phi) \geq \theta^\star$. Thus, MIFA applies the fixed eligibility rule only when requested by the search executor.

4.3.2 *Unified Best-First Execution.*

We first present execution over a fixed relation D and then extend it to appended tuples ΔD. Given a query $q = (\mathbf{q}_e, k, \phi)$, a fixed serving version $(\mathcal{J}, \pi^\star, \theta^\star)$, and $\epsilon \geq 0$, MIFA returns a buffer B of at most k predicted-eligible tuples ordered by $\preceq_q$. Exact mode ($\epsilon = 0$) returns $\widehat{\mathsf{Ans}}(q, D)$, whereas approximate mode ($\epsilon > 0$) may terminate early subject to a $(1 + \epsilon)$ distance-boundary guarantee.

State and coverage invariant. Each tuple in D belongs to exactly

one leaf of $\mathcal{T}_e$. MIFA maintains: (i) a buffer B containing the best k positively resolved tuples seen so far, or all such tuples if fewer than k (line 1); (ii) a unified priority queue Q of tree nodes and unresolved tuples (line 6); and (iii) a registration set S_{reg} recording tuples already finalized or scheduled for inference to prevent duplicate processing (line 1). At all times, every tuple not finalized or safely pruned is represented either explicitly by an unresolved-tuple entry or implicitly under an unexpanded node in Q.

When $|B| = k$, let t_B be its lowest-ranked tuple under $\preceq_q$. Define the current distance boundary and minimum queue key as

$$\tau_B = \begin{cases} \mathsf{dist}(\mathbf{q}_e, t_B[A_{\mathbf{e}}]), & |B| = k, \\ \infty, & |B| < k, \end{cases}$$

$$\lambda_Q = \begin{cases} \min_{x \in Q} \mathsf{key}(x), & Q \neq \emptyset, \\ \infty, & Q = \emptyset. \end{cases}$$

For a node N, $\mathsf{key}(N)$ is its safe geometric lower bound; for an unresolved tuple t, $\mathsf{key}(t) = \mathsf{dist}(\mathbf{q}_e, t[A_{\mathbf{e}}])$. Thus, λ_Q lower-bounds the distance of every unprocessed tuple represented by Q, while B always retains the best positively resolved tuples under $\preceq_q$.

ANN seeding and geometric traversal (lines 2–21). MIFA first retrieves seeds from $\mathcal{I}_e$ and applies only hard evaluation. A hard pass updates B, and both hard pass and fail outcomes are finalized and registered in S_{reg}. An unresolved seed is neither inferred nor registered; it remains covered by $\mathcal{T}_e$ and is processed when encountered during traversal. Thus, seeding does not affect exact-mode coverage or $\widehat{\mathsf{Ans}}(q, D)$; in approximate mode, it initializes B and tightens the boundary used for permitted early stopping.

MIFA initializes Q with the tree root, keyed by its safe lower bound. When it pops an internal node, it inserts each child N' unless B is full and $\mathsf{LB}(\mathbf{q}_e, N') > \tau_B$. The strict inequality retains subtrees that may contain a tuple tied at τ_B but ranked ahead of t_B under $\preceq_q$. When MIFA pops a leaf, it computes the exact distance of each unregistered tuple. The predicate $\mathsf{CanAffect}(t, B, q)$ holds if B is not full or t ranks ahead of t_B under $\preceq_q$. A tuple failing this test cannot enter the current or any subsequently improved buffer, so MIFA registers it without evaluating eligibility. For a competitive tuple, a hard pass updates B, a hard fail is discarded, and an unresolved outcome is inserted into Q at its exact distance. MIFA registers all three outcomes, preventing duplicate processing.

Deferred statistical inference (lines 22–25). When an unresolved tuple t reaches the queue head, MIFA rechecks whether it can still enter B, namely, whether B is not full or t ranks ahead of its lowest-ranked tuple under $\preceq_q$. MIFA discards a now-noncompetitive tuple without inference; otherwise, it applies the statistical stage of $\pi^\star$ using evidence from $\mathcal{I}_{\mathsf{mv}}$. For this statistical case, let $\widehat{e}(t, \phi) \in \{0, 1\}$ denote its predicted eligibility under the fixed deployment pair, where $\widehat{e}(t, \phi) = 1$ iff $z_r(t, \phi) \geq \theta^\star$. If $\widehat{e}(t, \phi) = 1$, MIFA inserts t into B and retains its best k tuples; otherwise, it discards t. Thus, MIFA invokes statistical inference only while its outcome can affect B and uses only safe geometric bounds, never predicted eligibility, to prune tree nodes.

Exact and approximate modes (lines 7–8). The parameter ϵ affects only termination; eligibility evaluation, exact-distance ranking, and safe node pruning remain unchanged.

Exact mode ($\epsilon = 0$). MIFA terminates when Q is empty or when B is full and $\lambda_Q > \tau_B$. The strict inequality ensures that all candidates at the current distance boundary are processed, including distance ties. Thus, B is the exact top-k over the fixed predicted-eligible set $\widehat{E}(q, D)$, namely $\widehat{\mathsf{Ans}}(q, D)$.

Approximate mode ($\epsilon > 0$). When B is full, MIFA may terminate early once $\tau_B \leq (1+\epsilon)\lambda_Q$. The returned k-th distance is then at most $1+\epsilon$ times the exact-mode k-th distance, although the returned tuple set may differ. If fewer than k tuples are predicted eligible, B never fills, so MIFA exhausts Q and returns all predicted-eligible tuples.

Base–delta maintenance. MIFA extends the fixed-relation procedure in Figure 3 to append-heavy workloads without changing its pruning, eligibility evaluation, or stopping rules.

Base and delta state. After each refresh, let D be the published base relation and ΔD the tuples appended since that refresh; the active searchable relation is $D \cup \Delta D$. The base tree $\mathcal{T}_e$ and delta tree $\Delta\mathcal{T}_e$ index these disjoint tuple sets. The delta tree reuses the frozen projection and split tests of $\mathcal{T}_e$ but maintains its own tuples and MBRs, providing safe bounds directly comparable with those of the base tree. For each append, MIFA inserts the tuple into ΔD and the ANN index (or an equivalent delta layer), routes it along one path in $\Delta\mathcal{T}_e$, and updates only that path's counts and MBRs. The published base tree remains unchanged throughout the serving period.

Unified search and refresh. To search $D \cup \Delta D$, MIFA retains the same B, Q, and S_{reg}. ANN seeding in lines 2–5 covers both relations, while line 6 initializes Q with both nonempty roots; all node and tuple entries then compete under the same ordering. The traversal, pruning, deferred inference, and stopping rules in lines 7–26 remain unchanged. Exact mode returns $\widehat{\mathsf{Ans}}(q, D \cup \Delta D)$, while approximate mode retains the $(1 + \epsilon)$ distance-boundary guarantee.

During a serving period, a delta tuple uses its corresponding frozen base-tree context and the current support index $\mathcal{I}_{\mathsf{mv}}$ (built from D_{tr}) for inference. It may be evaluated and returned but contributes no statistical support until refresh. An append becomes visible only after its ΔD, ANN, and delta-tree updates are published together, and each query uses one read-consistent version throughout execution. A periodic refresh merges $D \cup \Delta D$ into a new base, reruns structure–policy selection, rebuilds the compatible structures, recalibrates $\theta^\star$, and resets ΔD. MIFA publishes the new version atomically, while in-flight queries continue using their previous versions.

4.4 Execution Acceleration

Although the joint executor avoids unnecessary inference and search work, an unfused implementation may still materialize and transfer relation-sized arrays of distances, hard outcomes, and unresolved states across operators or the host–device boundary, even when the algorithm in Figure 3 examines only a small frontier. This residual movement can dominate query latency.

Basic idea. MIFA addresses this bottleneck with *residency-preserving candidate streams*. Instead of materializing distances and eligibility states for all tuples, it performs hard eligibility screening and distance competition where the vectors and scalar columns already reside. A compact candidate record crosses an operator or device boundary only if the tuple can still enter the current top-k buffer B. Accordingly, a hard failure produces no downstream state,

a hard pass enters distance competition directly, and an unresolved tuple triggers joint inference only after its distance shows that it can still affect B. Thus, the materialized state scales with the competitive search frontier rather than the relation, turning logical lazy inference into *output-sensitive materialization.*

CPU execution. Under a fixed serving version, the eligibility outcome for (t, ϕ) depends on the tuple and predicate, but not on the query embedding $\mathbf{q}_e$ or k. MIFA can therefore reuse resolved outcomes across queries sharing ϕ, caching them as postings of tuple row IDs that reference resident data. Workers stream IDs from the eligible postings, compute d_t directly from the resident vector $t[A_\mathbf{e}]$, and maintain local frontiers ordered lexicographically by $(d_t, \mathrm{id}(t))$, consistent with $\preceq_q$. Tuples without cached outcomes remain covered by unified traversal, and only the merged competitive prefix is forwarded downstream. This avoids per-query relation-sized distance/state arrays, corpus copies, and inter-operator state, limiting traffic to resident-vector reads and candidate-sized refinement state.

GPU execution. On GPUs, MIFA applies the same principle at the host–device boundary. It keeps vectors and predicate columns device-resident and fuses distance computation and hard evaluation with local candidate selection. It suppresses hard failures and emits pages of candidate records $(\mathrm{id}(t), d_t, h_t)$, ordered by $(d_t, \mathrm{id}(t))$, only while they may still affect B; here, h_t denotes the hard-evaluation outcome. If a query terminates after consuming M_q records, MIFA transfers only $O(M_q)$ candidate state to the host, rather than a $\Theta(|D|)$ distance/state array. It maintains a compact host frontier and invokes indexed joint inference only for unresolved candidates.

Multi-GPU execution. MIFA supports replication for throughput and partitioning for capacity. In replication mode, it places the full resident corpus on each GPU and assigns each query to one GPU, requiring no cross-device merge. In partitioned mode, it searches each corpus shard independently and stably merges the resulting local streams under the common $(d_t, \mathrm{id}(t))$ order. Both modes use the same fixed $(\pi^\star, \theta^\star)$ and preserve joint-inference semantics. The partitioned mode exchanges and merges only compact candidate pages rather than centrally scheduling resident tuples.

4.5 Guarantees and Complexity

This section states MIFA's correctness guarantees and summarizes its complexity; full proofs and derivations are deferred to [1].

Hard-decision soundness. If all constraints in Σ_{det} are sound, then every certified domain contains the true hidden value, and hard pass/fail decisions are consistent with the complete relation D^*.

Theorem 1: *For query q, let E_{pass} and E_{fail} be the tuples receiving hard* pass *and* fail*, respectively. Then $E_{\mathsf{pass}} \subseteq E^*(q, D^*)$ and $E_{\mathsf{fail}} \cap E^*(q, D^*) = \emptyset$.* □

A conjunctive predicate hard-passes only if every atom is observed or certified true, and hard-fails if any atom is observed or certified false. Thus, eligibility errors arise only from statistical estimation, limited support, or distribution shift.

Threshold monotonicity and selection. For the fixed deployed policy $\pi^\star$, increasing θ can only shrink the predicted-eligible set: for $\theta_1 \le \theta_2$, $\widehat{E}_{\pi^\star,\theta_2}(q, D) \subseteq \widehat{E}_{\pi^\star,\theta_1}(q, D)$. MIFA selects $\theta^\star$ by held-out calibration over a finite, predeclared set Θ. If the $m_{\mathsf{cal}} = |Q_{\mathsf{cal}}|$ calibration queries are independently drawn from the deployment distribution and their losses lie in $[0, 1]$, then, with probability at least $1 - \delta$, the expected loss of $\theta^\star$ exceeds that of the best candidate in Θ by at most $2\sqrt{\frac{\log(2|\Theta|/\delta)}{2m_{\mathsf{cal}}}}$. This bounds finite-sample threshold-selection error for the fixed $\pi^\star$, not errors in the underlying eligibility estimates or distribution shift; see Appendix B in [1].

Search guarantees relative to $\widehat{E}_{\pi^\star,\theta^\star}$. Let $\mathsf{Ans}_{\pi^\star,\theta^\star}(q, D)$ be the exact top-k over $\widehat{E}_{\pi^\star,\theta^\star}(q, D)$ under $\preceq_q$, and let $\widehat{\mathsf{Ans}}_{\pi^\star,\theta^\star,\epsilon}(q, D)$ be the output of Section 4.3 for $\epsilon \ge 0$.

Theorem 2: *Given relation D and the fixed deployment pair $(\pi^\star, \theta^\star)$,* MIFA *satisfies: (i) in exact mode, $\widehat{\mathsf{Ans}}_{\pi^\star,\theta^\star,0}(q, D) = \mathsf{Ans}_{\pi^\star,\theta^\star}(q, D)$; and (ii) in approximate mode ($\epsilon > 0$), if $|\widehat{E}_{\pi^\star,\theta^\star}(q, D)| \ge k$, then $\tau_B \le (1+\epsilon) d^{\mathsf{pred}}_{k,\pi^\star,\theta^\star}$, where $d^{\mathsf{pred}}_{k,\pi^\star,\theta^\star}$ is the distance of the kth-nearest tuple in $\widehat{E}_{\pi^\star,\theta^\star}(q, D)$. Consequently, every returned tuple has distance at most $(1+\epsilon) d^{\mathsf{pred}}_{k,\pi^\star,\theta^\star}$.* □

In exact mode, safe geometric bounds prevent pruning potential competitors, while the unified queue preserves the remaining search frontier. In approximate mode, the theorem provides a distance-boundary guarantee rather than exact top-k membership. These guarantees apply to the fixed predicted-eligible set, not to the accuracy of its eligibility decisions. If $\widehat{E}_{\pi^\star,\theta^\star}(q, D) = E^*(q, D^*)$, exact mode recovers the complete-data top-k.

Complexity. We summarize the cost below.

Online query time. Let n_{node} and n_{tuple} be the numbers of node-bound evaluations and tuple examinations, respectively. Let n_{queue} be the number of unresolved tuples inserted into the unified queue, $n_{\mathsf{inf}} \le n_{\mathsf{queue}}$ those invoking statistical inference, n_{buf} the number of buffer-update attempts, and H the maximum queue size. Let C_{seed} be the ANN seeding cost, and let c_{lb}, c_{dist}, c_{det}, and c_{inf} denote the per-operation costs of lower-bound evaluation, distance computation, hard evaluation, and statistical inference. The online query time is $O(C_{\mathsf{seed}} + n_{\mathsf{node}} c_{\mathsf{lb}} + (n_{\mathsf{node}} + n_{\mathsf{queue}}) \log(H+1) + n_{\mathsf{tuple}}(1 + c_{\mathsf{dist}} + c_{\mathsf{det}}) + n_{\mathsf{inf}} c_{\mathsf{inf}} + n_{\mathsf{buf}} \log(k+1))$. Thus, the cost depends on the explored frontier and competitive unresolved tuples, rather than requiring inference over the entire relation.

Space complexity. Let H be the maximum queue size and W_{inf} the size of the reusable inference workspace. Persistent and query-local space are $O(|D| + |\mathcal{I}_e| + |\mathcal{T}_e| + |\mathcal{I}_{\mathsf{mv}}| + |\pi^\star|)$ and $O(H + |S_{\mathsf{reg}}| + k + W_{\mathsf{inf}})$.

Append cost. Excluding ANN insertion, appending one tuple under the base–delta design costs $O(d d_p + d_p h_\Delta)$, where d and d_p are the original and projected dimensions, respectively, and h_Δ is the delta-tree height. The two terms correspond to embedding projection and delta-tree routing and path updates.

5 EXPERIMENTAL STUDY

We evaluated MIFA on vector–scalar hybrid datasets with missing scalar attributes along four dimensions: (1) answer quality across missingness and predicate workloads; (2) inference/search efficiency and scalability; (3) the contributions of hierarchical contexts, serving structures, and inference–search coordination; and (4) exact/approximate execution and incremental maintenance.

Dataset	Pred.	$\lvert D_{\text{tr}}\rvert$	$\lvert D\rvert$	$\lvert Q_{\text{test}}\rvert$	d	$\lvert\bar{\mathcal{A}}\rvert$
WANDS [6]	C+N	15,873	6,349	240	1,024	2
Books [26, 27]	C+N	4,000	1,600	68	1,024	3
MovieLens [13, 40]	C+N	6,606	2,642	300	1,024	4
Yelp [44]	C+N	29,725	11,890	300	1,024	4
H&M [32]	C	52,563	21,025	600	1,024	1
Amazon-multi [26]	C	13,397	5,359	300	1,024	2
AppReviews [17]	N	144,033	57,613	600	1,024	2
Amazon-C4 [26, 27]	C	30,000	12,000	1,000	1,024	1
ArXiv [15]	N	50,000	20,000	1,000	4,096	2
ArXiv-large [15]	N	50,000	2,735,264	1,000	128	2
Books-1M	N/C/C+N	4,000	1,000,000	64	128	3

Table 1: The tested datasets. C, N, and C+N denote categorical, numerical, and mixed categorical–numerical predicates.

5.1 Experimental Setting

Datasets and workloads. Table 1 summarizes ten real-world vector–scalar workloads and the controlled Books-1M scale-up. WANDS, Books, MovieLens, and Yelp use mixed predicates; H&M, Amazon-multi, and Amazon-C4 use categorical predicates; and AppReviews, ArXiv, and ArXiv-large use numerical predicates. Books-1M supports all three types. Dataset-specific query construction is detailed in Appendix D.1 of [1].

Data separation. We split a complete relation D^*_{full} by entity into disjoint partitions D^*_{tr}, D^*_{val}, D^*_{cal}, and D^*_{test}, and independently apply the same predeclared masking mechanism to each: $D_x = \text{Mask}(D^*_x)$. The masked test partition forms the searchable relation $D = D_{\text{test}}$. MIFA uses D_{tr} for template discovery and support construction, D_{val} to select $(\Gamma^\star, \pi^\star)$ and refit $\pi^\star$, and D_{cal} only to select $\theta^\star$. All methods receive the same masked partitions, development data, and applicable pre-mask supervision; hidden values never serve as evidence or tuple features.

Missingness. Let ρ be the incomplete-tuple rate, m the number of designated query-relevant attributes, and w the masked cells per incomplete tuple. By default, $w = 1$ and $m = 2$ where possible, with one numerical and one categorical attribute for mixed workloads; single-attribute workloads use $m = 1$. We evaluate three mechanisms: MCAR samples incomplete tuples uniformly; *historical candidate exposure* (HCE) assigns a β_{exp} fraction of the masking budget to tuples frequently appearing in historical embedding-only top-L lists; and *query-answer exposure* (QAE) assigns an α_{ans} fraction to complete-data answer tuples. Unless stated otherwise, $\rho = 20\%$, $w = 1$, $\beta_{\text{exp}} = 0.5$, and $\alpha_{\text{ans}} = 1$. All methods use identical masks; further details appear in Appendix D.1 of [1].

Queries. We use native benchmark queries when available; otherwise, we construct numerical, categorical, and mixed predicates from training-set quantiles and frequencies. Training queries form $\mathcal{Q}_{\text{hist}}$ and are shared by all workload-aware methods, while all methods use the same test queries. Further construction details appear in Appendix D.1 of [1].

Baselines. We compared MIFA with two classes of baselines.

(1) Missing-value handling. (a) Drop treats unresolved tuples as ineligible, whereas (b) MeanMode imputes missing numerical and categorical values using training-set means and modes, respectively. We further evaluate (c) MICE [39], (d) MissForest [38], (e) HoloClean [33], and (f) Raha+Baran [23, 24]. (g) We implement QueryCls, a learned baseline that trains a query-conditioned classifier on pseudo-missing tuple–predicate pairs using observed scalar values, masks, projected embeddings, and predicate encodings. To isolate missing-value handling, each method is followed by exact filtered top-k search over its repaired values or eligibility decisions.

Table 2: Overall F_1@10 under three missingness mechanisms ($\rho = 20\%$, $w = 1$, $k = 10$). Mean/MF/HC/R+B/QCls denote MeanMode/MissForest/HoloClean/ Raha+Baran/QueryCls; bold indicates the best result.

Dataset	Drop	Mean	MICE	MF	HC	R+B	QCls	MIFA
(A) Predicate-level MCAR								
WANDS	.803	.826	.825	.849	.837	.855	.863	**.912**
Books	.867	.840	.803	.877	.876	.900	.889	**.930**
MovieLens	.902	.855	.876	.899	.884	.900	.912	**.920**
H&M	.797	.804	.799	.942	.940	.903	.907	**.973**
Yelp	.880	.836	.874	.938	.925	.955	.912	**.965**
Amazon-multi	.838	.570	.851	.899	.881	.838	.882	**.906**
AppReviews	.793	.942	.935	.935	.793	.935	.958	**.961**
Average	.840	.811	.852	.906	.877	.898	.903	**.938**
(B) Historical candidate exposure (HCE; $\beta_{\text{exp}} = 0.5$)								
WANDS	.786	.818	.813	.833	.788	.773	.851	**.910**
Books	.772	.762	.694	.832	.776	.832	.775	**.895**
MovieLens	.925	.873	.889	.862	.901	.920	.932	**.934**
H&M	.768	.775	.769	.928	.927	.835	.907	**.970**
Yelp	.827	.791	.823	.951	.892	.936	.890	**.954**
Amazon-multi	.850	.533	.850	.891	.885	.847	.889	**.917**
AppReviews	.718	.934	.927	.929	.724	.877	.951	**.953**
Average	.807	.784	.823	.889	.842	.860	.885	**.933**
(C) Query-answer exposure (QAE; $\alpha_{\text{ans}} = 1$)								
WANDS	.000	.153	.153	.287	.416	.568	.546	**.826**
Books	.000	.210	.214	.493	.017	.634	.113	**.791**
MovieLens	.194	.388	.358	.387	.407	.206	.671	**.811**
H&M	.143	.179	.154	.745	.708	.614	.879	**.960**
Yelp	.001	.222	.134	.799	.266	.444	.544	**.864**
Amazon-multi	.000	.228	.017	.333	.038	.025	.463	**.515**
AppReviews	.000	.788	.765	.760	.000	.738	.913	**.922**
Average	.048	.310	.256	.543	.264	.461	.590	**.813**

(2) Filtered vector search. (a) FAVOR [36] and (b) SIEVE [19] support all predicate types; (c) iRangeGraph [43] supports numerical ranges; and (d) UNG [3] supports categorical labels; we extend it to numerical ranges by mapping values and ranges to labels and label sets, respectively. Each workload includes only methods supporting its predicate type, with other combinations reported as N/A. Each applicable static backend is paired with Drop and HoloClean. For numerical-range workloads, we additionally include (e) DSG [30] and (f) WoW [41].

Fairness and metrics. All methods share data splits, masks, and test queries, with no test data used for tuning. Applicable filtered-ANN backends are tuned to the same validation search-quality target. Repair methods use only masked observations; Raha+Baran additionally receives its recommended default budget of labeled training cells. MIFA uses exact mode unless varied, and its exact outputs are verified against brute-force top-k over the same predicted-eligible set. Our primary metric is $F_1@k$ against the complete-data answers $\text{Ans}^*(q, D^*_{\text{test}})$; we also report eligibility diagnostics and applicable offline, materialization, online, and amortized costs.

Parameters and implementation. Unless stated otherwise, we set $k = 10$, $\epsilon = 0$, and $L = 10k$; missingness defaults are given above. We predeclare $\Theta = \{0, 0.01, \ldots, 0.99, 1\}$ and choose $\theta^\star$ on D_{cal} by minimizing the query-level objective $1 - F_1@k$. We use HNSW [25] for ANN seeding and run five repetitions on a 104-core Intel Xeon Gold 5320 server (2.20 GHz) with 504 GB RAM and two NVIDIA GeForce RTX 4090 GPUs; within each repetition, all methods share the same masks and queries.

5.2 Exp-1: Effectiveness

Overall effectiveness. We evaluated MIFA's effectiveness in recovering complete-data filtered top-k answers against Drop, MeanMode, MICE, MissForest, HoloClean, Raha+Baran, and QueryCls. MIFA runs in exact mode, while each baseline applies exact filtered top-k search to its repaired relation or predicted-eligible set. This iso-

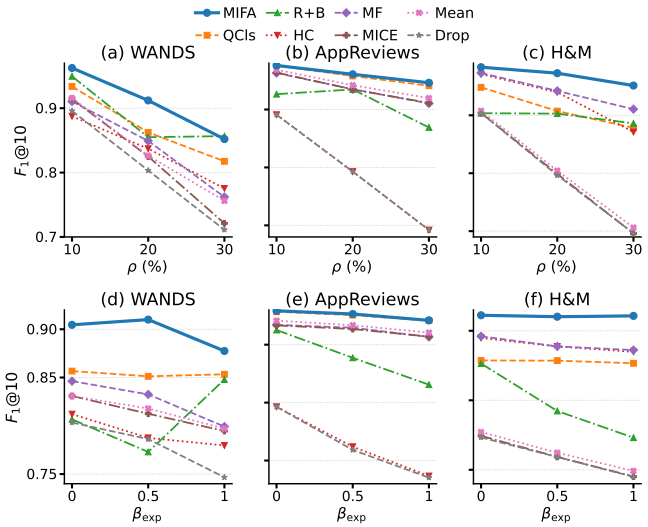


Figure 4: Varying missingness with ρ and β_{exp}.

lates missing-value handling from ANN approximation; corresponding combinations with filtered-ANN backends (*e.g.,* FAVOR, SIEVE, iRangeGraph, and UNG) are therefore omitted. We tested seven workloads under MCAR, HCE, and QAE, with $\rho = 20\%$, $w = 1$, and $k = 10$. Under the same incomplete-tuple budget, these mechanisms shift missingness from uniformly sampled tuples to historically competitive candidates and then to complete-data answer tuples, progressively increasing its likelihood of affecting top-k membership.

As shown in Table 2, MIFA achieves the highest F_1 across all workloads. Its average F_1 is 0.938, 0.933, and 0.813 under MCAR, HCE, and QAE, respectively, with corresponding gains over MICE/HoloClean/Raha+Baran of 8.62/6.14/4.01, 10.99/9.14/7.34, and 55.62/54.82/35.13 points. Thus, MIFA maintains high quality under missingness and gains more as uncertainty becomes decision-relevant, supporting its strategy of retaining top-k-competitive unresolved candidates for joint and context-adaptive inference.

Varying missingness. We evaluated MIFA's robustness as missingness becomes more prevalent or more concentrated among competitive candidates. Under MCAR, we varied $\rho \in \{10\%, 20\%, 30\%\}$ with $w = 1$. Under HCE, we fixed (ρ, w) and varied $\beta_{\text{exp}} \in \{0, 0.5, 1\}$ while preserving per-attribute masking budgets and reporting the realized exposure among historically competitive candidates. Results are averaged across three workloads.

As shown in Figure 4, increasing ρ from 10% to 30% reduces MIFA's average F_1 by 5.7 points (0.974→0.917), compared with 7.4 points for QueryCls, the strongest baseline (0.953→0.879). Accordingly, MIFA's advantage over QueryCls widens from 2.1 to 3.8 points. Across the β_{exp} sweep, MIFA maintains an average F_1 of 0.930–0.945, exceeding the strongest baseline by 3.1–4.1 points. These results show that MIFA degrades gracefully as missingness increases or shifts toward competitive candidates, supporting its strategy of retaining unresolved candidates for joint inference using query-conditioned, location-aware support.

Varying predicate workload. We evaluated MIFA's robustness to predicate complexity and result-set size by varying predicate type, conjunction arity, and k. The main comparison uses each dataset's native single-attribute predicates; arity sweeps use datasets with sufficient applicable attributes and match selectivity where possible.

Table 3: F_1 under varying predicate workloads with exact search and HCE ($\rho = 20\%$, $\beta_{\text{exp}} = 0.5$, $w = 1$). Bold and underlined values indicate the best and second-best results.

Setting	Drop	MeanMode	MICE	MissForest	HoloClean	Raha+Baran	QueryCls	MIFA
A. Predicate type — WANDS (F_1@10)								
Numerical	**0.926**	**0.926**	**0.926**	0.923	**0.926**	0.729	**0.926**	**0.926**
Categorical	0.886	0.886	0.862	0.872	0.873	0.874	0.886	**0.912**
Mixed	0.796	0.734	0.826	0.852	0.800	0.836	0.794	**0.909**
B. Conjunction arity — Books (F_1@10)								
$\lvert\phi\rvert = 1$	0.926	0.926	0.883	0.876	0.883	0.954	0.926	**0.984**
$\lvert\phi\rvert = 2$	0.856	0.850	0.784	0.873	0.854	0.910	0.856	**0.933**
$\lvert\phi\rvert = 3$	0.754	0.712	0.735	0.822	0.781	0.845	0.745	**0.883**
C. Requested result size — WANDS (F_1@k)								
$k = 5$	0.795	0.823	0.813	0.806	0.773	0.773	0.841	**0.898**
$k = 10$	0.786	0.818	0.813	0.833	0.787	0.773	0.851	**0.910**
$k = 20$	0.797	0.831	0.831	0.858	0.821	0.782	0.869	**0.931**
$k = 50$	0.816	0.852	0.852	0.880	0.853	0.809	0.890	**0.940**

Table 4: Efficiency on `ArXiv-large` at $d = 128$ under HCE ($\rho = 20\%$, $\beta_{\text{exp}} = 0.5$). IRG for iRangeGraph, HC for HoloClean.

Method	Handler	F_1@10 ↑	Eng. Rec. ↑	P50 (ms) ↓	P95 (ms) ↓	QPS ↑	Index (MiB) ↓	RAM (GiB) ↓	Build (s) ↓
FAVOR	Drop	0.568	0.841	141.191	155.361	7.3	767.0	9.286	91.96
FAVOR	HC	0.568	0.841	113.059	127.984	8.9	767.0	9.334	95.69
SIEVE	Drop	0.662	**1.000**	30.655	41.121	36.1	21110.0	322.688	217.64
SIEVE	HC	0.662	**1.000**	30.537	41.600	35.7	33067.3	327.635	208.63
IRG	Drop	0.661	0.996	1.898	2.267	525.8	1800.5	8.886	671.74
IRG	HC	0.660	0.996	1.812	2.069	549.7	1800.4	8.980	747.57
UNG	Drop	0.675	0.958	3.529	204.283	16.1	3494.2	**3.575**	467.20
UNG	HC	0.696	0.959	3.692	205.710	16.0	3694.6	3.774	522.94
DSG	Drop	0.662	1.000	3.952	7.081	225.8	5739.4	21.642	4192.72
DSG	HC	0.662	1.000	6.450	7.959	155.1	5739.4	21.640	4160.97
WoW	Drop	0.661	0.999	6.473	8.853	150.4	2183.5	13.250	316.82
WoW	HC	0.661	0.999	6.982	9.447	138.3	2183.5	13.439	612.90
MIFA	Native	**0.799**	**1.000**	**0.935**	**1.090**	**1108.6**	**257.7**	8.908	**50.57**

Table 5: Mean online latency (ms/query) of MIFA-CPU; "–" denotes stages merged into fused kernels.

	WANDS (240 Q)		H&M (600 Q)		AppReviews (600 Q)	
Component	Steady	Cold	Steady	Cold	Steady	Cold
Candidate / shortlist	0.139	0.418	–	–	1.138	1.140
Controller + final top-k	0.157	0.185	0.135	0.139	0.283	0.269
Exact-distance evaluation	0.825	1.020	3.141	3.161	10.782	10.043
Direct checks	0.102	0.245	0.195	0.212	0.951	0.951
Support / joint inference	0.004	5.618	0.008	0.059	0.009	1.074
Cache lookup	0.164	0.346	–	–	0.175	0.298
Total	**1.390**	**7.831**	**3.478**	**3.571**	**13.338**	**13.775**

All sweeps use default HCE and exact execution; we also report support coverage and statistical-resolution rates.

As shown in Table 3, MIFA's advantage grows with predicate complexity, reaching 5.71 points for mixed predicates and remaining 3.77 points at $|\phi| = 3$. It also maintains a 4.94–6.29 point advantage across $k = 5$–50. These results show that joint conjunction evaluation reduces compounding errors under complex predicates while remaining robust across result-set sizes.

We further evaluated MIFA across predicate selectivity, historical-evidence size, eligibility thresholds, and attribute-skewed, value-dependent, tuple-block, and oracle answer-targeted missingness; see Appendix D.2 in [1] for details.

5.3 Exp-2: Efficiency

Efficiency. We evaluated end-to-end efficiency on a 1M-row searchable subset of ArXiv-Large (2.76M rows) under default HCE. We compared MIFA with all applicable filtered-ANN baselines, each paired with Drop and HoloClean.

As shown in Table 4, MIFA achieves 1.090 ms P95 latency and 1108.6 queries per second (QPS), providing a 1.90× latency speedup and 2.02× higher throughput than the fastest external CPU configuration, iRangeGraph +HoloClean. Its 257.7-MiB auxiliary index is 3.0–128.3× smaller than the external indexes, its 8.9-GiB peak

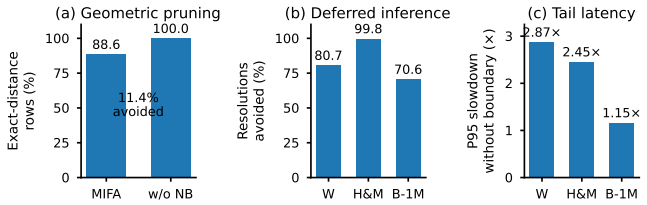


Figure 5: Pruning power of MIFA.

Table 6: Scalability on nested unique ArXiv-large **prefixes at** $d = 128$ **under HCE. Baselines are served with** HoloClean**.**

Method	20K				100K				1M			
	P50	P95	QPS	Index	P50	P95	QPS	Index	P50	P95	QPS	Index
FAVOR	2.068	2.369	503.6	**15.3**	9.060	9.713	117.1	**76.7**	113.059	127.984	8.9	767.0
SIEVE	0.287	0.396	3722.7	427.0	2.322	3.116	472.1	2811.1	30.537	41.600	35.7	33067.3
iRangeGraph	0.574	0.661	1771.8	23.0	1.370	1.914	702.2	143.1	1.812	2.069	549.7	1800.4
UNG	0.223	2.775	1133.9	21.7	0.283	11.348	286.1	114.6	3.692	205.710	16.0	3694.6
DSG	0.451	0.561	2279.3	92.3	1.070	1.344	955.0	513.7	6.450	7.959	155.1	5739.4
WoW	0.429	0.503	2350.4	36.7	2.070	2.591	479.7	195.0	6.982	9.447	138.3	2183.5
MIFA-CPU	**0.209**	**0.237**	4821.1	59.4	**0.282**	**0.343**	3439.7	83.0	**0.935**	**1.090**	1108.6	**257.7**
MIFA-1GPU	1.334	1.434	6435.4	59.4	1.365	1.481	6129.3	83.0	1.191	1.618	3462.1	**257.7**
MIFA-2GPU	1.348	1.457	**12245.9**	59.4	1.406	1.594	**11913.6**	83.0	1.277	1.787	**6106.4**	**257.7**

RSS is comparable to iRangeGraph, and its index construction is 1.82× faster than the fastest external method. These gains do not sacrifice quality: MIFA achieves $F_1@10 = 0.799$, 13.7 points above the strongest external pipeline. This demonstrates the efficiency of its fused execution, which resolves missing-predicate eligibility only when a competitive tuple can affect the top-k boundary.

Execution breakdown. Table 5 profiles MIFA on WANDS (mixed), H&M (categorical), and AppReviews (numerical predicates) in both steady-state and cold-cache serving. Steady-state serving reuses populated support and inference state, whereas cold-cache serving includes first-touch construction.

In steady state, exact-distance evaluation accounts for 59.3–90.3% of latency; support retrieval and joint inference contribute at most 0.3%. Lightweight candidate generation, control, deterministic checks, and cache operations account for the remainder. Cold-cache overhead is small on H&M and AppReviews, but increases WANDS latency from 1.390 to 7.831 ms, mainly due to 5.618 ms of first-touch support and inference work. Once populated, this work falls to at most 0.009 ms. That is, after a workload-dependent initialization cost, MIFA follows a distance-dominated steady-state path with minimal inference and control overhead.

Pruning power. We evaluated two complementary sources of MIFA's efficiency: partition-tree bounds prune noncompetitive subtrees before exact-distance evaluation, while the evolving top-k boundary prunes examined tuples before statistical inference.

As shown in Figure 5, subtree pruning avoids 11.4% of exact-distance evaluations on H&M, while tuple-boundary pruning avoids 70.6–99.8% of eager statistical resolutions across the three workloads. Disabling tuple-boundary pruning increases P95 latency by 1.15–2.87×. MIFA combines index-level geometric pruning with executor-level eligibility pruning to reduce distance/inference work.

Scalability. We evaluated scalability using nested 20K-, 100K-, and 1M-row prefixes of ArXiv-Large under HCE. As shown in Table 6, MIFA-CPU achieves P50/P95 latencies of 0.209/0.237, 0.282/0.343, and 0.935/1.090 ms, respectively, providing 1.67–3.92× P95 speedups over the fastest external baseline. Its auxiliary index grows modestly from 59.4 to 257.7 MiB. Reusable support state and compact candidate processing limit predicate-specific work to competitive unresolved tuples. Two GPU replicas further achieve 1.76–1.94× the throughput of one GPU and reach 6106.4 QPS at 1M rows. Thus,

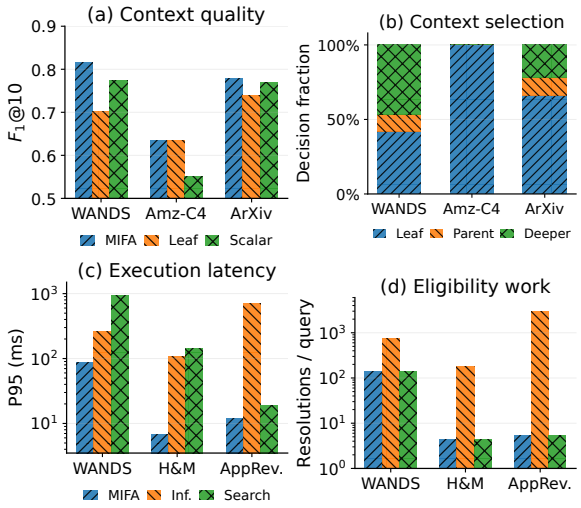


Figure 6: Hierarchical-context and execution-order ablations.

Dataset	MIFA	$-\mathcal{I}_e$	$-\mathcal{T}_e$	$-\mathcal{I}_{\text{mv}}$	$-\mathcal{J}$
WANDS	0.816 / 54.65	0.816 / 48.73	0.775 / 48.61	0.816 / 332.63	0.775 / 129.15
Amazon-C4	0.634 / 6.91	0.634 / 5.99	0.552 / 3.83	0.634 / 21.29	0.552 / 3.25
ArXiv	0.779 / 20.88	0.779 / 20.30	0.769 / 6.97	0.779 / 949.31	0.769 / 29.16
AppReviews	0.929 / 5.27	0.929 / 3.15	0.884 / 27.26	0.929 / 31.25	0.884 / 21.18

Table 7: Serving-structure ablation across four workloads. Each entry reports $F_1@10$**/P95 latency (ms).** MIFA$_{-\mathcal{I}_e}$ **and** MIFA$_{-\mathcal{I}_{\text{mv}}}$ **preserve** MIFA**'s eligibility policy, whereas** MIFA$_{-\mathcal{T}_e}$ **and** MIFA$_{-\mathcal{J}}$ **use** ScalarOnly **inference.**

MIFA maintains low CPU tail latency and modest index growth as the corpus expands while scaling throughput with additional GPUs.

We further evaluated efficiency under varying workload characteristics and across all datasets, construction and memory breakdowns, and scalability with embedding dimension and scalar-schema size; see Appendix D.3 in [1] for details.

5.4 Exp-3: Ablation Study

Hierarchical location-aware contexts. We evaluated hierarchical backoff and embedding locality using two variants. LeafOnly uses the same scalar and embedding-local evidence as MIFA but abstains when leaf support is insufficient, whereas ScalarOnly removes embedding-location conditions from the support templates.

As shown in Figure 6(a–b), backoff is most beneficial when leaf support is sparse. MIFA improves $F_1@10$ over LeafOnly by 11.50 and 4.05 points on WANDS and ArXiv, where only 42.0% and 66.3% of its decisions, respectively, stop at the leaf. On Amazon-C4, all decisions use leaf contexts, so the two methods match. Removing embedding locality with ScalarOnly lowers $F_1@10$ on all three workloads, by up to 8.25 points. Thus, hierarchical backoff addresses sparse local support, while embedding locality excludes neighborhood-inconsistent evidence.

Serving-structure ablation. We ablated $\mathcal{J} = (\mathcal{I}_e, \mathcal{T}_e, \mathcal{I}_{\text{mv}})$. MIFA$_{-\mathcal{I}_e}$ omits ANN seeding and relies on tree traversal; MIFA$_{-\mathcal{T}_e}$ scans the searchable relation and uses ScalarOnly; MIFA$_{-\mathcal{I}_{\text{mv}}}$ scans D_{tr} for the same support; and MIFA$_{-\mathcal{J}}$ scans both searchable and training relations using scalar-only inference. Only MIFA$_{-\mathcal{I}_e}$ and MIFA$_{-\mathcal{I}_{\text{mv}}}$ preserve MIFA's eligibility policy. For isolation, all use the same fused CPU backend with the hard-pass direct-top-k shortcut disabled.

Table 7 shows that replacing $\mathcal{I}_{\text{mv}}$ with scans preserves F_1 but in-

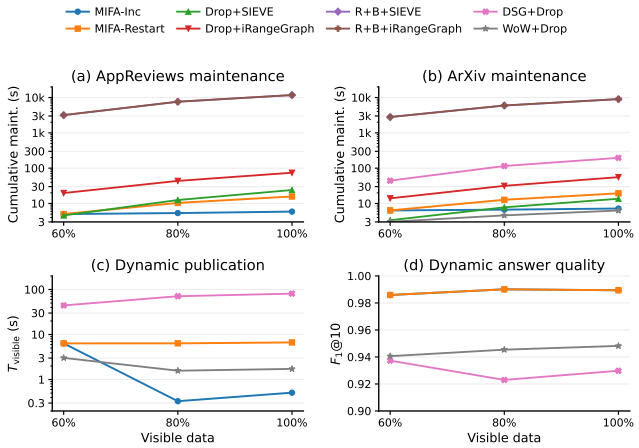


Figure 7: Incremental-ingestion performance as the visible fraction of each stream increases from 60% to 100%.

Method	F_1@10 ↑	P95 (ms) ↓	$T_{visible}$ (s) ↓	Cum. maint. (s) ↓
D+SIEVE	0.716	8.39	11.53	24.06
D+iRG	0.699	1.17	30.58	74.16
RB+SIEVE	0.549	10.02	4132.36	11723.60
RB+iRG	0.539	1.01	4144.40	11757.59
DSG+D	0.470	**0.14**	91.80	209.06
WoW+D	0.553	0.22	4.41	17.84
MIFA-R	0.756	1.03	5.41	15.78
MIFA-I	**0.763**	1.76	**0.53**	**5.88**

Table 8: Incremental-serving on AppReviews**.** MIFA-**R and** MIFA-**I denote restart and incremental maintenance.**

creases P95 by 3.08–45.46×, while removing $\mathcal{T}_e$ eliminates location-aware inference and lowers F_1 by up to 8.25 points. Removing $\mathcal{I}_e$ preserves quality and is slightly faster, showing that ANN seeding is not the main efficiency source at these scales. Thus, the support index provides the largest serving-time gain, whereas the projected tree supplies local evidence for accurate inference. Moreover, on the same AppReviews corpus, MIFA uses only 2.33 GiB peak host RSS versus 294.1 GiB for Raha+Baran, avoiding relation-wide repair.

Execution-order ablation. We isolated scheduling effects by comparing MIFA with InferenceFirst and SearchFirst under the same fixed $(\pi^\star, \theta^\star)$, support evidence, and exact semantics. InferenceFirst resolves all eligibility before search, SearchFirst orders all exact distances before resolving eligibility, and MIFA interleaves the two. We report P95 latency and statistical resolutions.

As shown in Figure 6(c–d), all methods achieve identical F_1, while MIFA provides 2.96–58.84× and 1.55–21.24× P95 speedups over InferenceFirst and SearchFirst, respectively. It avoids 81.5–99.8% of InferenceFirst's statistical resolutions; although SearchFirst performs similar eligibility work, it constructs the complete distance order first. By tightening top-k boundary and inference on-demand, MIFA avoids unnecessary inference and distance ordering.

5.5 Exp-4: Approximation and Maintenance

Incremental ingestion. We evaluated whether MIFA's base–delta design supports fast incomplete-data updates while preserving quality and reducing maintenance. On AppReviews and ArXiv, we build an initial version from the earliest 60% of timestamp-ordered tuples and append batches to reach 80% and 100% visibility. MIFA-Incremental keeps the base immutable and updates only the ANN delta state and $\Delta\mathcal{T}_e$, whereas MIFA-Restart rebuilds all searchable

Data	ϵ	F_1	P95	Node/q	Dist./N	Res./q	R-Rec.	Max-d_k
H&M	0	0.979	75.79	1882.5	0.813	38.27	1.000	1.0000
	0.005	0.975	22.19	246.8	0.072	39.82	0.995	1.0046
ArXiv	0	0.807	155.60	2047.0	0.998	16.03	1.000	1.0000
	0.0025	0.806	91.46	1750.0	0.774	18.36	0.988	1.0024

Table 9: Approximate execution on H&M **and** ArXiv**.**

structures. Both freeze $\pi^\star$, $\theta^\star$, $\mathcal{I}_{\text{mv}}$, and the projection between refreshes; appended tuples are immediately searchable but contribute statistical support only after refresh. We compare with {Drop, Raha+Baran}×{SIEVE, iRangeGraph} restart pipelines and, for numerical-range streams, DSG and WoW paired with Drop.

As shown in Figure 7, at 100% visibility, MIFA-Incremental reduces cumulative maintenance by 2.68–2.69× over MIFA-Restart, 1.89–12.60× over Drop-based rebuilds, and 1251–1998× over the measured Raha+Baran repair–rebuild pipelines. Table 8 further shows that, on AppReviews, it publishes the final append batch 173.2× faster than DSG and 8.32× faster than WoW, while improving final F_1 by 29.3 and 21.0 points, respectively. These gains arise because MIFA updates only newly arrived searchable state rather than repeatedly repairing and indexing the active snapshot. Although the dynamic indexes offer lower standalone latency, they assume decidable range predicates. Overall, the base–delta design provides fast publication, restart-level quality, and substantially lower long-run maintenance.

Approximate execution. We compared exact mode ($\epsilon = 0$) with predeclared positive ϵ values, reporting F_1, P95 latency, expanded nodes, exact-distance work, statistical resolutions, exact-mode result-set recall, and the maximum $\tau_B / d^{\text{pred}}_{k,\pi^\star,\theta^\star}$. The last two measure membership loss and verify the distance bound, respectively. Queries with fewer than k predicted-eligible tuples are reported separately because both modes exhaust the frontier.

As shown in Table 9, on H&M, $\epsilon = 0.005$ reduces P95 latency, expanded nodes, and exact-distance work by 70.7%, 86.9%, and 91.2%, with only a 0.47-point F_1 loss and 0.995 recall. On ArXiv, $\epsilon = 0.0025$ reduces P95 latency and distance work by 41.2% and 22.4%, with a 0.08-point F_1 loss and 0.988 recall. The savings mainly reflect earlier frontier termination; statistical resolutions remain comparable or increase modestly. The maximum boundary ratios are 1.0046 and 1.0024, with no observed violations of the corresponding $(1+\epsilon)$ bounds. Thus, approximation substantially reduces geometric work with little membership loss.

6 CONCLUSION

This paper formalizes *incomplete filtered ANN search* and presents MIFA, which answers such queries without materializing missing values. MIFA combines sound hard decisions with context-adaptive estimation of unresolved predicate conjunctions, supported by projected hierarchies, a shared support index, and unified best-first execution that applies inference only to distance-competitive tuples. We establish hard-decision soundness, a finite-sample threshold-selection guarantee, exact execution over the fixed predicted-eligible set, and a $(1+\epsilon)$ approximate distance-boundary guarantee. Experiments on real-world hybrid workloads demonstrate improved $F_1@k$ and query efficiency over drop-missing and first-impute-then-query baselines, while a base–delta design supports append-heavy ingestion between periodic policy–index refreshes.

Future work includes richer predicates, fully dynamic maintenance, and adaptation to abrupt distribution or missingness shifts.

REFERENCES

[1] 2026. Code, datasets and full version. https://anonymous.4open.science/r/MIFA_Release-68D6.

[2] George Beskales, Mohamed A. Soliman, and Ihab F. Ilyas. 2008. Efficient Search for the Top-k Probable Nearest Neighbors in Uncertain Databases. *Proceedings of the VLDB Endowment* 1, 1 (2008), 326–339. https://doi.org/10.14778/1453856.1453895

[3] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6, Article 246 (2024), 27 pages. https://doi.org/10.1145/3698822

[4] José Cambronero, John K. Feser, Micah J. Smith, and Samuel Madden. 2017. Query Optimization for Dynamic Imputation. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1310–1321. https://doi.org/10.14778/3137628.3137641

[5] Hilari Caminal, Yannis Chronis, Fatma Ozcan, Yannis Papakonstantinou, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-art and Research Opportunities. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5488–5500. https://doi.org/10.14778/3747329.3747331

[6] Yan Chen, Shujian Liu, Zheng Liu, Weiyi Sun, Linas Baltrunas, and Benjamin Schroeder. 2022. WANDS: Dataset for Product Search Relevance Assessment. In *Advances in Information Retrieval: 44th European Conference on IR Research, ECIR 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13185)*. Springer, 128–141. https://doi.org/10.1007/978-3-030-99736-6_9

[7] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-Art and Research Opportunities. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5488–5492. https://doi.org/10.14778/3750601.3750700

[8] Michele Dallachiesa, Amr Ebaid, Ahmed Eldawy, Ahmed Elmagarmid, Ihab F. Ilyas, Mourad Ouzzani, and Nan Tang. 2013. NADEEF: A Commodity Data Cleaning System. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 541–552. https://doi.org/10.1145/2463676.2463715

[9] Nilesh Dalvi and Dan Suciu. 2007. Efficient Query Evaluation on Probabilistic Databases. In *Proc. VLDB*.

[10] Wenfei Fan, Floris Geerts, Jianzhong Li, and Ming Xiong. 2010. Discovering conditional functional dependencies. *IEEE Transactions on Knowledge and Data Engineering* 23, 5 (2010), 683–698.

[11] Siddharth Gollapudi, Harsha Vardhan Simhadri, Raghavendra Srinivasa, B. Manohar, Anum Das, Biswa Bhattacharjee, and A. Nandan. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference (WWW)*. 3406–3416. https://doi.org/10.1145/3543507.3583419

[12] Paolo Guagliardo and Leonid Libkin. 2017. A Formal Semantics of SQL Queries, Its Validation, and Applications. *Proc. VLDB Endow.* 11, 1 (2017), 27–39. https://doi.org/10.14778/3151113.3151116

[13] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4, Article 19 (2015), 19 pages. https://doi.org/10.1145/2827872

[14] Yupeng Hou, Jiacheng Li, Xiangjun Fu, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation: Benchmarking LLMs as Semantic Encoders. *arXiv preprint arXiv:2403.03952* (2024). arXiv:2403.03952 [cs.IR]

[15] Patrick Iff, Paul Bruegger, Marcin Chrapek, Maciej Besta, and Torsten Hoefler. 2025. Benchmarking Filtered Approximate Nearest Neighbor Search Algorithms on Transformer-based Embedding Vectors. *arXiv preprint arXiv:2507.21989* (2025). arXiv:2507.21989 [cs.DB]

[16] Tomasz Imielinski and Witold Lipski Jr. 1984. Incomplete Information in Relational Databases. *J. ACM* 31, 4 (1984), 761–791. https://doi.org/10.1145/1634.1886

[17] Guoxin Kang, Zhongxin Ge, Jingpei Hu, Xueya Zhang, Lei Wang, and Jianfeng Zhan. 2025. BigVectorBench: Heterogeneous Data Embedding and Compound Queries are Essential in Evaluating Vector Databases. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1536–1550. https://doi.org/10.14778/3718057.3718078

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.

[19] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4723–4736.

[20] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proceedings of the VLDB Endowment* 18, 4 (2024), 1118–1130. https://doi.org/10.14778/3717755.3717770

[21] Leonid Libkin. 2015. SQL's Three-Valued Logic and Certain Answers. In *Proceedings of the 18th International Conference on Database Theory (ICDT)*. https://doi.org/10.4230/LIPIcs.ICDT.2015.331

[22] Yiming Lin and Sharad Mehrotra. 2023. ZIP: Lazy Imputation during Query Processing. *Proceedings of the VLDB Endowment* 17, 1 (2023), 28–40. https://doi.org/10.14778/3617838.3617841

[23] Mohammad Mahdavi and Ziawasch Abedjan. 2020. Baran: Effective Error Correction via a Unified Context Representation and Transfer Learning. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1948–1961. https://doi.org/10.14778/3407790.3407801

[24] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data*. 865–882. https://doi.org/10.1145/3299869.3324956

[25] Yu A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2018). https://doi.org/10.1109/TPAMI.2018.2889473

[26] McAuley Lab. 2023. Amazon Reviews 2023. https://amazon-reviews-2023.github.io/. Dataset; cited with the BLaIR paper [14].

[27] McAuley Lab. 2024. Amazon-C4. https://huggingface.co/datasets/McAuley-Lab/Amazon-C4. Complex product search dataset built based on Amazon Reviews 2023; cited with the BLaIR paper [14].

[28] James Jie Pan, Jianguo Wang, and Guoliang Li. 2023. Survey of Vector Database Management Systems. *arXiv preprint arXiv:2310.14021* (2023).

[29] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3, Article 120 (2024), 27 pages. https://doi.org/10.1145/3654923

[30] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3256–3268. https://doi.org/10.14778/3748191.3748193

[31] Neoklis Polyzotis, Spiros Skiadopoulos, Panos Vassiliadis, Alkis Simitsis, and Nils-Erik Frantzell. 2007. Supporting Streaming Updates in an Active Data Warehouse. In *ICDE*. 476–485.

[32] Qdrant Team. 2023. ANN Filtered Retrieval Datasets. https://github.com/qdrant/ann-filtering-benchmark-datasets. Filtered vector-similarity benchmark datasets, including the H&M clothing workload; accessed 2026-08-20.

[33] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1190–1201. https://doi.org/10.14778/3137628.3137631

[34] Uwe Röhm, Klemens Böhm, Hans-Jörg Schek, and Heiko Schuldt. 2002. FAS – A Freshness-Sensitive Coordination Middleware for a Cluster of OLAP Components. In *VLDB*. 754–765.

[35] Harsh Sehgal, Mohammad Sadek, Ismail Şengör Altingöz, and Günes Şalihoglu. 2025. NaviX: Similarity Search with Filtering in Large-Scale Vector Databases. *Proc. VLDB Endow.* 18, 11 (2025), 4438–4450.

[36] Junjie Song, Yu Liu, Guoyu Hu, Zhongle Xie, Ming Yang, Beng Chin Ooi, and Ke Zhou. 2026. FAVOR: Efficient Filter-Agnostic Vector ANNS Based on Selectivity-Aware Exclusion Distances. *Proceedings of the ACM on Management of Data* 4, 3, Article 183 (2026), 25 pages. https://doi.org/10.1145/3802060

[37] Shaoxu Song and Lei Chen. 2011. Differential dependencies: Reasoning and discovery. *ACM Transactions on Database Systems (TODS)* 36, 3 (2011), 1–41.

[38] Daniel J. Stekhoven and Peter Bühlmann. 2012. MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics* 28, 1 (2012), 112–118. https://doi.org/10.1093/bioinformatics/btr597

[39] Stef van Buuren and Karin Groothuis-Oudshoorn. 2011. mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software* 45, 3 (2011), 1–67. https://doi.org/10.18637/jss.v045.i03

[40] Jesse Vig, Shilad Sen, and John Riedl. 2012. The Tag Genome: Encoding Community Knowledge to Support Novel Interaction. *ACM Transactions on Interactive Intelligent Systems* 2, 3, Article 13 (2012), 44 pages. https://doi.org/10.1145/2362394.2362395

[41] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 6, Article 378 (2025), 27 pages. https://doi.org/10.1145/3769843

[42] Garrett Wolf, Hemal Khatri, Bhaumik Chokshi, Yi Chen, and Subbarao Kambhampati. 2007. Query Processing over Incomplete Autonomous Databases. In *Proceedings of the 33rd International Conference on Very Large Data Bases*. 651–662.

[43] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-Dedicated Graphs for Range-Filtering Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6, Article 239 (2024), 26 pages. https://doi.org/10.1145/3698814

[44] Yelp Inc. [n. d.]. Yelp Open Dataset. https://business.yelp.com/data/resources/open-dataset/. Dataset; accessed 2026-06-18.

[45] Jiaxu Zhu, Jiayu Yuan, Kaiwen Yang, Xiaobao Chen, Shihuan Yu, Hongchang Lv, Yan Li, and Bolong Zheng. 2025. An Experimental Evaluation of Hybrid Querying on Vectors. *Proceedings of the VLDB Endowment* 19, 2 (2025), 183–195.

A VALIDATION-BASED JOINT DESIGN SELECTION

This appendix details candidate structure–policy construction and cross-fitted evaluation, the joint selection of $(\Gamma^\star, \pi^\star)$, and the subsequent policy refitting and threshold calibration described in Section 4.2.3.

Target and body enumeration. For each predicate and predeclared missingness shape represented in $\mathcal{Q}_{\mathsf{val}}$, MIFA enumerates the set Y of predicate attributes that may remain unresolved after hard evaluation. Duplicate residual sets are represented once. For each candidate Y, the dependency-discovery procedure in Section 4.2.1 searches D_{tr} for scalar bodies $X_s \subseteq \bar{\mathcal{A}} \setminus Y$ and matching conditions that identify training tuples informative about Y. A training tuple can support a candidate only when all attributes in $X_s \cup Y$ are observed.

For every retained (X_s, Y) pair, MIFA generates a scalar-only template and a location-aware counterpart $r : X_s, [\mathsf{Loc}(A_{\mathbf{e}})] \rightsquigarrow Y$. It also considers a body-free template ($X_s = \emptyset$) for each target as a fallback when no observed scalar body applies. Templates with insufficient training support are discarded. The resulting finite template set is $\mathcal{T}_{\mathsf{cand}}$.

Candidate designs and validation data. A candidate structural configuration is $\Gamma = (d_p, B_{\mathsf{leaf}})$, where d_p specifies the projection dimension and B_{leaf} the leaf capacity. A candidate policy $\pi \in \mathcal{P}_{\mathsf{cand}}$ specifies an ordered subset of $\mathcal{T}_{\mathsf{cand}}$, together with its applicability and priority rules, minimum-support and shrinkage parameters, hierarchical and scalar backoff sequences, and score-map fitting procedures. At this stage, it contains neither fitted deployment score maps nor concrete serving-tree nodes.

The finite candidate family combines the predicate and missingness shapes in $\mathcal{Q}_{\mathsf{val}}$, the enumerated templates, the predeclared matching and backoff families, and the hyperparameter grids in $\mathcal{H}_{\mathsf{val}}$. MIFA considers each template's admissible matching levels and discards templates and policies that fail the predeclared template- or shape-level coverage requirements.

Pseudo-missing validation instances. MIFA derives labeled tuple–predicate instances $v = (t, \phi, y)$ from originally observed validation outcomes, where $y \in \{0, 1\}$ indicates whether t satisfies the complete predicate. It masks selected observed predicate values to produce a fixed pseudo-missing snapshot $\widetilde{D}_{\mathsf{val}}$. The masked values are retained only as validation labels and never used as inference evidence, tuple features, or support. Natural missingness remains unchanged, and instances whose complete outcomes remain indeterminate are excluded.

The masks, instances, validation queries, and snapshot are fixed before candidate evaluation and shared by every design. For each Γ, MIFA constructs a temporary hierarchy and support index from D_{tr} and evaluates the design over $\widetilde{D}_{\mathsf{val}}$. Only training tuples with observed template bodies and targets provide statistical support. Let C denote the resulting finite set of admissible structure–policy pairs.

To evaluate score maps without reusing their fitting instances, MIFA partitions the unresolved validation instances $\mathcal{V}_{\mathsf{val}}$ into F fixed folds that respect the entity or temporal boundaries of the outer split. For an instance v in fold $f(v)$, it fits each candidate score map on the other $F - 1$ folds and evaluates it on v. Thus, validation labels select designs and fit score maps but never enter the support index.

Predicate-inference quality. For each $v \in \mathcal{V}_{\mathsf{val}}$, let

$$y_v = \mathbf{1}[\phi_v(v^\star) = \mathsf{true}]$$

be its ground-truth joint-satisfaction outcome, and let $\widetilde{p}^{\mathsf{cf}}_{\Gamma,\pi}(v) \in [0, 1]$ be the cross-fitted score produced by (Γ, π). For validation loss, an inference-stage abstention receives score zero while remaining distinguishable from a statistical rejection in the deployed policy.

MIFA defines

$$\mathsf{InfLoss}(\Gamma, \pi) = \frac{1}{|\mathcal{V}_{\mathsf{val}}|} \sum_{v \in \mathcal{V}_{\mathsf{val}}} \left(\widetilde{p}^{\mathsf{cf}}_{\Gamma,\pi}(v) - y_v\right)^2$$

and

$$\mathsf{InfScore}(\Gamma, \pi) = 1 - \mathsf{InfLoss}(\Gamma, \pi).$$

This score evaluates joint predicate satisfaction rather than reconstruction of individual missing values. Together with the coverage requirements, it penalizes unstable local estimates and excessive backoff or abstention.

End-to-end replay cost. Because the deployment threshold is selected later on calibration data, validation does not optimize a threshold on D_{val}. Instead, MIFA fixes a finite representative threshold set Θ_{rep} before validation and replays the exact online algorithm for every $q \in \mathcal{Q}_{\mathsf{val}}$ and $\vartheta \in \Theta_{\mathsf{rep}}$. Each replay uses candidate policy π, threshold ϑ, and the temporary hierarchy and support index built under Γ. A validation-only ANN seed index is fixed across candidates because it is independent of Γ and π. Every unresolved tuple uses the score map fitted without its fold, so both inference quality and replay cost are evaluated out of fold.

For one replay, let $m_{q,\vartheta}$ be the number of evaluated node bounds; $n_{q,\vartheta}$ the number of tuple-identifier examinations; $n^{\mathsf{dist}}_{q,\vartheta}$ and $n^{\mathsf{det}}_{q,\vartheta}$ the numbers of exact-distance and deterministic predicate evaluations; $u^{\mathsf{queue}}_{q,\vartheta}$ the number of unresolved tuples inserted into the queue; $u^{\mathsf{infer}}_{q,\vartheta} \le u^{\mathsf{queue}}_{q,\vartheta}$ the number that invoke statistical inference; $b_{q,\vartheta}$ the number of buffer-update attempts; and $H_{q,\vartheta}$ the maximum queue size. The replay cost is

$$\begin{aligned} T_{\mathsf{rep}}(q, \vartheta; \Gamma, \pi) = {} & C_{\mathsf{seed}}(q) + m_{q,\vartheta} c_{\mathsf{lb}} \\ & + (m_{q,\vartheta} + u^{\mathsf{queue}}_{q,\vartheta}) c_Q \log(H_{q,\vartheta} + 1) \\ & + n_{q,\vartheta} c_{\mathsf{reg}} + n^{\mathsf{dist}}_{q,\vartheta} c_v \\ & + n^{\mathsf{det}}_{q,\vartheta} c_{\mathsf{det}} + u^{\mathsf{infer}}_{q,\vartheta} c_{\mathsf{inf}} \\ & + b_{q,\vartheta} c_B \log(k + 1). \end{aligned}$$

Here, C_{seed} includes only backend candidate retrieval, whereas c_{inf} includes indexed support retrieval, hierarchical backoff, joint-count estimation, shrinkage, and score-map evaluation. MIFA estimates the unit costs once by microbenchmarking and reuses them across candidates.

The average replay cost is

$$\overline{T}_{\mathsf{rep}}(\Gamma, \pi) = \frac{1}{|\mathcal{Q}_{\mathsf{val}}||\Theta_{\mathsf{rep}}|} \sum_{q \in \mathcal{Q}_{\mathsf{val}}} \sum_{\vartheta \in \Theta_{\mathsf{rep}}} T_{\mathsf{rep}}(q, \vartheta; \Gamma, \pi),$$

which MIFA normalizes over the fixed candidate set:

$$\mathsf{TimeCost}(\Gamma,\pi) = \frac{\overline{T}_{\mathsf{rep}}(\Gamma,\pi)}{\max_{(\Gamma',\pi')\in C}\overline{T}_{\mathsf{rep}}(\Gamma',\pi')}.$$

If all replay costs are zero, it sets $\mathsf{TimeCost}(\Gamma,\pi) = 0$ for every candidate. The proxy already captures projected-distance pruning because tighter bounds reduce node visits, distance computations, and unresolved tuples reaching the inference frontier; MIFA therefore adds no separate pruning reward.

Joint selection and final refitting. Given a predeclared trade-off parameter $\lambda \geq 0$, MIFA defines

$$\mathsf{Utility}(\Gamma,\pi) = \mathsf{InfScore}(\Gamma,\pi) - \lambda \cdot \mathsf{TimeCost}(\Gamma,\pi)$$

and selects

$$(\Gamma^\star,\pi^\star) = \arg\max_{(\Gamma,\pi)\in C} \mathsf{Utility}(\Gamma,\pi).$$

It resolves ties by first choosing the lexicographically smallest (d_p, B_{leaf}) and then the lexicographically smallest candidate policy. A larger λ favors lower online cost, whereas a smaller value emphasizes held-out inference quality.

At selection time, $\pi^\star$ fixes the retained templates, applicability and priority rules, backoff sequences, support and shrinkage parameters, and score-map fitting procedures, but not the final score-map parameters. MIFA discards all temporary hierarchies, support indexes, and fold-specific maps; no node or split from a validation hierarchy is retained.

Under $\Gamma^\star$, MIFA constructs the final partition topology and split tests from D_{tr}. It routes the searchable snapshot through this fixed topology to populate the leaf contents and searchable-descendant MBRs, forming $\mathcal{T}_e$, and builds $\mathcal{I}_e$ over the searchable embeddings. It also records the training tuples' contexts and builds the persistent support index $\mathcal{I}_{\mathsf{mv}}$ from D_{tr}. MIFA then routes all pseudo-missing validation instances through the final hierarchy, reapplies $\pi^\star$, retrieves support exclusively from D_{tr}, and refits each selected score map on all statistically resolved instances assigned to its template. These refitted maps complete the fixed deployment policy $\pi^\star$.

Calibration and publication. MIFA routes each masked calibration tuple through the fixed hierarchy while retaining D_{tr} as the sole support source. The hidden calibration values are used only to compute the predeclared query-level loss under exact scan and ranking. Calibration tuples enter neither the persistent search structures nor $\mathcal{I}_{\mathsf{mv}}$ and modify no MBR.

The threshold set Θ is fixed before observing the independent calibration query groups. MIFA selects $\theta^\star$ without changing $\Gamma^\star$, the persistent structures, $\pi^\star$, or its score maps. It then atomically publishes the searchable snapshot, $\mathcal{I}_e$, $\mathcal{T}_e$, $\mathcal{I}_{\mathsf{mv}}$, $\pi^\star$, and $\theta^\star$ as one serving version.

Between refreshes, the base and delta trees jointly cover the active searchable relation. Appended tuples use the frozen base split tests for inference-context assignment but provide no support to $\mathcal{I}_{\mathsf{mv}}$ until refresh. A refresh rebuilds and atomically publishes a mutually compatible structure–policy version.

B PROOFS OF THE THEORETICAL GUARANTEES

All proofs below fix the read-consistent relation snapshot and compatible structure–policy version pinned at query start.

Proof of Theorem 1.

Proof: An observed atom is evaluated against its correct value and is therefore sound. For a missing attribute A, whenever deterministic evaluation forms a nonempty certified domain $V(t,A)$, the soundness assumption gives

$$t^*[A] \in V(t,A).$$

If every value in $V(t,A)$ satisfies the atom, then the hidden value $t^*[A]$ satisfies it as well. If $V(t,A)$ is disjoint from the atom's satisfying region, then $t^*[A]$ violates the atom. A partially overlapping domain, an empty domain produced by conflicting evidence, or the absence of an applicable constraint yields no hard decision.

A hard pass requires every conjunct in ϕ to be certified true, whereas a hard fail requires at least one conjunct to be certified false. Consequently,

$$E_{\mathsf{pass}} \subseteq E^*(q,D^*), \qquad E_{\mathsf{fail}} \cap E^*(q,D^*) = \emptyset,$$

which proves the claim. □

Proof of the threshold-selection bound in Section 4.5.

Proof: Condition on the fixed masked calibration snapshot $\widetilde{D}_{\mathsf{cal}}$, its hidden labels in D_{cal}, the deployment policy Π, and the finite candidate set Θ, where Π and Θ are fixed before the calibration query groups are observed. For each $\vartheta \in \Theta$, define

$$\ell_\vartheta(q_i) = L(q_i;\Pi,\vartheta,D_{\mathsf{cal}}).$$

Because the calibration query groups $q_1,\ldots,q_{m_{\mathsf{cal}}}$ are i.i.d. and $L \in [0,1]$, the variables

$$\ell_\vartheta(q_1),\ldots,\ell_\vartheta(q_{m_{\mathsf{cal}}})$$

are i.i.d. and bounded in $[0,1]$. Hoeffding's inequality therefore gives, for every fixed $\vartheta \in \Theta$,

$$\Pr\left[\left|\widehat{R}(\vartheta) - R(\vartheta)\right| > \alpha\right] \leq 2e^{-2m_{\mathsf{cal}}\alpha^2}.$$

A union bound over Θ, with

$$\alpha = \sqrt{\frac{\log(2|\Theta|/\delta)}{2m_{\mathsf{cal}}}},$$

shows that

$$\left|\widehat{R}(\vartheta) - R(\vartheta)\right| \leq \alpha$$

holds simultaneously for every $\vartheta \in \Theta$ with probability at least $1-\delta$.

Let

$$\vartheta^\star \in \arg\min_{\vartheta\in\Theta} R(\vartheta).$$

On the preceding event, the empirical minimizer θ satisfies

$$\begin{aligned} R(\theta) &\leq \widehat{R}(\theta) + \alpha \\ &\leq \widehat{R}(\vartheta^\star) + \alpha \\ &\leq R(\vartheta^\star) + 2\alpha. \end{aligned}$$

Substituting the definition of α yields

$$R(\theta) \leq \min_{\vartheta\in\Theta} R(\vartheta) + 2\sqrt{\frac{\log(2|\Theta|/\delta)}{2m_{\mathsf{cal}}}},$$

which proves the claim. The declared tie-breaking rule for empirical minimizers does not affect this bound. □

Proof of exact execution in Theorem 2.

Proof: Call a tuple *finalized* once its fixed policy decision has been obtained, including an abstention decision. At every point in the execution, B contains the best

$$\min\{k, \text{number of finalized predicted-eligible tuples}\}$$

finalized tuples under the complete order $\preceq_q$. The registration set S_{reg} prevents duplicate processing: it records tuples finalized during ANN seeding or tree traversal and unresolved tuples already represented by pending tuple entries.

We first establish the safety of both pruning rules. If no full buffer is ever formed, neither rule discards a tuple for lack of competitiveness, so there is nothing to prove. Otherwise, once the buffer becomes full, it remains full. Consider a state s with a full buffer B_s, and let

$$w_s = \mathsf{worst}_q(B_s), \qquad \tau_s = \mathsf{dist}(\mathbf{q}_e, w_s[A_\mathbf{e}]).$$

Because every subsequent buffer retains the best k finalized eligible tuples seen so far, its worst tuple can only improve under $\preceq_q$. In particular, if w is the final worst buffer tuple, then

$$w \preceq_q w_s \qquad \text{and} \qquad \tau_B \le \tau_s.$$

If a node N is pruned at state s, then

$$\mathsf{LB}(\mathbf{q}_e, N) > \tau_s.$$

Every descendant t of N therefore has distance strictly greater than τ_s and cannot precede w_s, hence cannot precede the no-worse final tuple w. In contrast, a tuple-level pruning decision uses the complete order rather than a strict distance comparison. If a tuple t is discarded because it cannot affect B_s, then

$$w_s \preceq_q t.$$

By transitivity,

$$w \preceq_q w_s \preceq_q t,$$

so t cannot improve the final buffer. This argument also covers a tuple whose distance equals τ_s but whose identifier-based tie-breaking position is worse than that of w_s.

The traversal maintains the following coverage invariant: every tuple that has not been finalized or safely pruned is represented in Q by either a pending tuple entry or an unexpanded node entry whose subtree contains it. The invariant holds initially because the active roots jointly cover every tuple not already finalized during ANN seeding. Popping an internal node replaces it with every child that is not safely pruned. Processing a leaf examines each unregistered tuple and either finalizes it, safely prunes it, or inserts it into Q as an unresolved tuple entry. When such a tuple entry is later popped, it is finalized unless the updated buffer makes it noncompetitive, in which case the tuple-level argument above applies. Thus, the invariant is preserved.

First suppose that $Q = \emptyset$ at termination. By the coverage invariant, every tuple has been finalized or safely pruned. The buffer contains the exact top-k among all finalized predicted-eligible tuples, while the preceding boundary argument shows that no safely pruned tuple can improve it. Therefore,

$$B = \mathsf{Ans}_{\Pi,\theta}(q, D).$$

Now suppose that $Q \neq \emptyset$. Exact termination then requires $|B| = k$ and

$$\lambda_Q > \tau_B,$$

where λ_Q is the smallest remaining queue key. Assume, for contradiction, that an omitted predicted-eligible tuple t should precede the worst tuple in B under $\preceq_q$.

If t has already been finalized, this contradicts the invariant that B stores the top-k finalized predicted-eligible tuples. If t has been safely pruned, the corresponding node- or tuple-level argument above shows that it cannot improve B.

Otherwise, let x_t be the queued tuple entry or queued ancestor representing t. For a node entry,

$$\mathsf{key}(x_t) \le \mathsf{dist}(\mathbf{q}_e, t[A_\mathbf{e}]),$$

whereas equality holds for a tuple entry. Because any tuple that precedes the current worst buffer tuple has distance at most τ_B, including a tuple that wins only through tie-breaking,

$$\lambda_Q \le \mathsf{key}(x_t) \le \mathsf{dist}(\mathbf{q}_e, t[A_\mathbf{e}]) \le \tau_B.$$

This contradicts $\lambda_Q > \tau_B$. Thus,

$$B = \mathsf{Ans}_{\Pi,\theta}(q, D),$$

or equivalently,

$$\widehat{\mathsf{Ans}}_{\Pi,\theta,0}(q, D) = \mathsf{Ans}_{\Pi,\theta}(q, D).$$

This proves exactness relative to the fixed predicted-eligible set. □

Proof of approximate execution in Theorem 2.

Proof: Assume first that

$$|\widehat{E}_{\Pi,\theta}(q, D)| \ge k.$$

If approximate execution terminates because $Q = \emptyset$, the proof of exact execution applies directly. Therefore,

$$\tau_B = d^{\text{pred}}_{k,\Pi,\theta}.$$

Otherwise, approximate termination requires $|B| = k$ and

$$\tau_B \le (1 + \epsilon)\lambda_Q.$$

Because B contains k predicted-eligible tuples,

$$d^{\text{pred}}_{k,\Pi,\theta} \le \tau_B.$$

Because B maintains the top-k finalized predicted-eligible tuples, every finalized predicted-eligible tuple outside B has distance at least τ_B. For a tuple pruned with a node, its distance is strictly greater than the full-buffer boundary at the pruning time. For a tuple pruned directly under $\preceq_q$, its distance is at least that boundary, with equality allowed when its tie-breaking position is worse. Since full-buffer boundaries are non-increasing, every safely pruned tuple therefore has distance at least the final τ_B. Finally, every predicted-eligible tuple remaining in the frontier is represented by an entry x satisfying

$$\lambda_Q \le \mathsf{key}(x) \le \mathsf{dist}(\mathbf{q}_e, t[A_\mathbf{e}]).$$

Thus, every predicted-eligible frontier tuple has distance at least λ_Q.

If $\lambda_Q < \tau_B$, the worst tuple in B has distance $\tau_B > \lambda_Q$. Consequently, at most $k-1$ predicted-eligible tuples have distance strictly below λ_Q, and hence

$$d^{\text{pred}}_{k,\Pi,\theta} \ge \lambda_Q.$$

Approximate termination then gives

$$\tau_B \le (1+\epsilon)\lambda_Q \le (1+\epsilon)d^{\text{pred}}_{k,\Pi,\theta}.$$

If $\lambda_Q \ge \tau_B$, no predicted-eligible tuple outside B has distance below τ_B. Since the worst tuple in B has distance τ_B,

$$d^{\text{pred}}_{k,\Pi,\theta} = \tau_B,$$

which also implies

$$\tau_B \le (1+\epsilon)d^{\text{pred}}_{k,\Pi,\theta}.$$

Combining the two cases yields

$$d^{\text{pred}}_{k,\Pi,\theta} \le \tau_B \le (1+\epsilon)d^{\text{pred}}_{k,\Pi,\theta}.$$

Because every tuple returned in B has distance at most τ_B, every returned tuple has distance at most

$$(1+\epsilon)d^{\text{pred}}_{k,\Pi,\theta}.$$

If

$$|\widehat{E}_{\Pi,\theta}(q,D)| < k,$$

the buffer cannot become full. The approximate stopping condition therefore cannot trigger before queue exhaustion. By the coverage invariant, every predicted-eligible tuple is then finalized and returned. □

C COMPLEXITY DETAILS

This appendix expands the complexity discussion deferred from Section 4.5.

Online complexity. Let m_v be the number of node-bound evaluations during traversal, including children subsequently pruned; n_v the number of tuple-identifier examinations through ANN seeding and leaf traversal, counting repeats; u_q the number of unresolved tuples inserted into the unified queue; $u_v \le u_q$ the number that remain competitive when popped and invoke statistical inference; b_v the number of buffer-update attempts; and H the maximum queue size.

Let c_{lb}, c_v, c_{det}, and c_{inf} be the costs of one node-bound computation, exact-distance evaluation, direct and deterministic hard evaluation, and statistical policy evaluation, respectively. Here, c_{inf} includes template selection, indexed support retrieval, hierarchical backoff, joint counting, shrinkage, and score-map evaluation. Let C_{seed} include only ANN candidate retrieval; exact-distance and predicate processing for retrieved candidates are counted in n_v.

Assuming expected $O(1)$ registration-set operations, the online query cost is

$$O\big(C_{\text{seed}} + m_v c_{\text{lb}} + (m_v + u_q)\log(H+1) + n_v(1 + c_v + c_{\text{det}}) + u_v c_{\text{inf}} + b_v \log(k+1)\big).$$

This bound is conservative: noncompetitive tuples avoid direct/deterministic hard evaluation, and unresolved tuples that become noncompetitive before reaching the queue head avoid statistical inference.

Space complexity. Let n be the number of searchable tuples, $N_{\mathcal{T}} = O(n)$ the number of active tree nodes, $n_{\text{tr}} = |D_{\text{tr}}|$, and M_{tr} the number of indexed observed scalar cells in D_{tr}. Let s_Π be the stored size of the fixed deployment policy, including its retained deterministic evidence, selected templates, applicability and priority rules, backoff sequences, support and shrinkage parameters, and score maps.

Beyond the original relation and plug-in ANN index, persistent space is

$$O\big(n + d_p N_{\mathcal{T}} + dd_p + n_{\text{tr}} + M_{\text{tr}} + s_\Pi\big).$$

The terms account for leaf tuple identifiers, projected MBRs and split metadata, the projection matrix, training-tuple context locations, the shared support index, and the stored deployment policy.

Let r_v be the number of tuple identifiers in S_{reg}, where $r_v \le \min\{n, n_v\}$, and let w_{inf} be the maximum temporary workspace of one statistical policy evaluation. Because inference workspaces can be reused, query-local space is

$$O(H + r_v + k + w_{\text{inf}}).$$

Append and refresh costs (base–delta). Let h_Δ be the length of a root-to-leaf path in the active delta tree. Excluding backend-specific ANN insertion, each append requires

$$O(dd_p + d_p h_\Delta)$$

work: the first term projects the new embedding, while the second routes the tuple and updates counts and MBRs along its path. Because the delta tree reuses the serving version's frozen projection and split tests, append processing performs no online repartitioning. Offline policy selection, validation replay, final refitting, and threshold calibration occur only when constructing or refreshing a published version and are excluded from the online bounds.

D ADDITIONAL EXPERIMENTAL DETAILS AND RESULTS

D.1 Detailed Experimental Configurations

Fixed settings and candidate grids. Table 10 reports the default settings and predeclared candidate grids used in our experiments. All candidate sets are fixed before validation. Dataset-specific choices are selected using D_{val} only, after which the deployment policy is refitted and θ is selected using D_{cal} as described in Appendix A. No test query, hidden test value, or test result is used to select any configuration.

The admissible template families, applicability rules, and hierarchical and scalar backoff orders are those defined in Sections 4.2.1 and 4.2.3. For every dataset, we report the configuration selected on D_{val}, including the retained templates, selected support and shrinkage parameters, projection dimension, leaf capacity, and ANN search parameters.

The following details expand the dataset, query, and missingness construction summarized in Section 5.1.

Dataset and query construction. WANDS retains its product-search queries and adds categorical and numerical metadata predicates. Books combines Amazon-C4 query embeddings with Amazon Reviews 2023 metadata predicates; MovieLens, Yelp, and Amazon – multi use held-out content queries with their corresponding metadata. ArXiv retains its benchmark query vectors with numerical range/equality filters, while Amazon – C4 retains its categorical filtered workload. AppReviews and H&M retain their native compound-numerical and categorical workloads, respectively, and Books – 1M reuses the Books queries. For constructed predi-

Setting	Value or candidate set
Default k	10
Default incomplete-tuple rate ρ	20%
Default missing attributes m	2 when applicable; 1 for native single-attribute workloads
Masked cells per incomplete tuple w	1
Candidate region size L	$10k$
Exposure fraction β_{exp}	0.5; varied over $\{0, 0.5, 1\}$
Oracle answer exposure γ_{ans}	$\{0.25, 0.5, 0.75, 1\}$ (appendix only)
Projection dimension d_p	$\{8, 16\}$
Leaf capacity B_{leaf}	$\{64, 128\}$
Minimum support $s_{\min}$	$\{10, 30, 50\}$
Shrinkage strength κ_r	$\{0, 10, 30\}$
Template and shape coverage	0.80/0.80
Representative thresholds Θ_{rep}	$\{0.25, 0.50, 0.75\}$
Eligibility thresholds Θ	$\{0, 0.01, \ldots, 0.99, 1\}$
ANN seed size	64
Approximation values ϵ	$\{0, 0.0025, 0.005, 0.01, 0.02, 0.05, 0.075, 0.10\}$
Historical-evidence fractions	{25%, 50%, 75%, 100%}
Raha + Baran annotation budget	20 training-cell labels
Queries per dataset	64–1,000 test queries; dataset-specific (Table 1)
Worker threads	8; 32 for the optimized Books − 1M CPU path
Independent repetitions	5

Table 10: Default settings and predeclared candidate grids.

cates, attributes are fixed from the schema and training workload, and constants use training quantiles or frequencies.

Data and query separation. Complete tuples are partitioned by entity into mutually disjoint D_{tr}, D_{val}, D_{cal}, and D^*_{test}, or chronologically for timestamped datasets, before controlled masking. Query sources are disjoint from searched tuples and partitioned by source entity or time into training, validation, calibration, and test groups; benchmarks without source entities are partitioned directly before masking and configuration. Training groups form $\mathcal{Q}_{\text{hist}}$. For the bound in Section 4.5, $\mathcal{Q}_{\text{cal}}$ contains one independently sampled query per source group; when a group contributes multiple queries, their losses are averaged so m_{cal} counts independent groups. Hidden scalar values are used only for labels and losses and never for support, features, configuration, or query construction.

Missingness budgets. For each partition D_s, the incomplete-tuple budget is $B_s = \text{round}(\rho|D_s|)$. When $m = 2$, the designated attributes split this budget approximately evenly over disjoint tuple sets; thus, ρ remains the overall incomplete-tuple rate and, at the default $w = 1$, every incomplete tuple has one masked cell. Mixed workloads designate one numerical and one categorical attribute; single-type workloads designate two attributes of that type when available. All designated attributes are fixed using only the schema and training workload.

MCAR samples each attribute's budget uniformly. HCE assigns a β_{exp} fraction by exposure-weighted sampling from embedding-only top-L candidates of training queries and fills the remainder uniformly; its exact construction is given below under *Historical candidate-exposure masks*. QAE instead allocates an α_{ans} fraction of the same budget to tuples in complete-data filtered top-k answers $G_s(q)$, masking a designated attribute referenced by the corresponding query, and fills any remainder uniformly. In the overall comparison, MCAR, HCE, and QAE share $\rho = 20\%$ and $w = 1$; HCE uses $\beta_{\text{exp}} = 0.5$, and QAE uses $\alpha_{\text{ans}} = 1$. All methods receive the same realized masks.

Controlled repetitions. Within each repetition, all methods receive the same complete-data snapshot, query sources, controlled masks, and train–validation– calibration–test partitions. For MCAR, the missing-rate experiments use nested masks, so a tuple masked at a lower value of ρ remains masked at every larger value. For historical candidate exposure, all methods use the same training-query workload and the same seeded weighted-sampling draws at each β_{exp}. Unless explicitly varied, the query set, predicate distribution, k, missingness mechanism, and random seed are held fixed. External backends are configured using their validation-selected operating points and run with the same number of worker threads. For every reported exact-mode result, we additionally verify equality with brute-force top-k over the fixed predicted-eligible set $\widehat{E}_{\pi^\star,\theta}(q, D)$.

Historical candidate-exposure masks. Let $\mathcal{Q}_{\text{hist}}(A)$ contain the training queries whose predicates reference a designated missing attribute A. For each partition D_s and query q, we rank only the tuples in D_s by embedding distance and let $C_L^{(s)}(q)$ denote its embedding-only top-L candidates. For tuple $t_i \in D_s$, we compute

$$s_s(i, A) = \sum_{q \in \mathcal{Q}_{\text{hist}}(A)} \mathbf{1}[t_i \in C_L^{(s)}(q)].$$

An exposure-guided fraction β_{exp} of A's preallocated missingness budget is sampled without replacement with probability proportional to $s_s(i, A)$; if the positive-score pool is insufficient, the remainder is filled uniformly. The rest of the attribute budget is also sampled uniformly from unselected tuples. When $m = 2$, the two attribute-specific tuple sets are kept disjoint, so every incomplete tuple still has exactly one masked cell. This construction uses only the fixed training-query workload, observed embeddings, and predicate-attribute identities; it never uses test queries, test answers, predicate outcomes, hidden scalar values, or method outputs.

Dataset-specific configurations. Table 11 reports the deployed structure–policy configuration for each workload. Structure and policy choices are selected on validation data, after which the retained policy is refitted and the eligibility threshold $\theta^\star$ is selected on the disjoint calibration split.

The selected configurations are compact across workloads: all policies use $s_{\min} = 10$ without additional shrinkage, while dataset variation is absorbed mainly by the projection width, template composition, and calibrated eligibility threshold. The controlled Books − 1M scale workload uses the same design procedure and selects a smaller leaf capacity than the corresponding Books workload.

D.2 Additional Effectiveness Results

Alternative missingness patterns. At the default incomplete-tuple rate ρ, we compare the standard MCAR setting with attribute-skewed, tuple-block, and value-dependent missingness. All patterns use the same complete test snapshot and test queries in each repetition. We keep the total incomplete-tuple budget $B_s = \text{round}(\rho|D_s|)$ fixed across patterns and report the resulting masked-cell rate separately, since tuple-block missingness may mask multiple cells in one incomplete tuple.

Attribute-skewed masks retain $w = 1$ but allocate the fixed to-

Dataset	d_p	B_{leaf}	Ord.	Support context X_s, $[\text{Loc}(A_e)]$	Target Y	m_{cal}	$\theta^\star$
WANDS	8	128	1	$[\text{Loc}(A_e)]$	{average_rating}	48	0.40
			2	{rating_count, subcategory}, $[\text{Loc}(A_e)]$	{product_class}		
			3	$[\text{Loc}(A_e)]$	{product_class}		
Books	8	128	1	{rating_number, subcategory}, $[\text{Loc}(A_e)]$	{average_rating}	10	0.81
			2	{subcategory}, $[\text{Loc}(A_e)]$	{category}		
MovieLens	8	128	1	$[\text{Loc}(A_e)]$	{avg_rating}	50	0.41
			2	$[\text{Loc}(A_e)]$	{primary_genre}		
Yelp	8	128	1	$[\text{Loc}(A_e)]$	{stars}	50	0.59
			2	{city}, $[\text{Loc}(A_e)]$	{state}		
H&M	16	128	1	{garment_group_name, section_name}, $[\text{Loc}(A_e)]$	{department_name}	100	0.30
Amazon − multi	8	128	1	{store}, $[\text{Loc}(A_e)]$	{main_category}	50	0.66
			2	$[\text{Loc}(A_e)]$	{store}		
AppReviews	8	128	1	$[\text{Loc}(A_e)]$	{review_length}	100	0.53
			2	$[\text{Loc}(A_e)]$	{star_rating}		
Amazon − C4	16	128	1	$[\text{Loc}(A_e)]$	{category}	200	0.33
ArXiv	8	128	1	{main_category, number_of_main_categories}, $[\text{Loc}(A_e)]$	{number_of_sub_categories}	100	0.44
			2	{license, main_category}, $[\text{Loc}(A_e)]$	{update_date}		
			3	$[\text{Loc}(A_e)]$	{update_date}		
Books − 1M	8	64	1	{rating_number, subcategory}, $[\text{Loc}(A_e)]$	{average_rating}	10	0.53

Table 11: Dataset-specific MIFA structures, ordered retained support templates, and calibrated eligibility thresholds. All selected templates use $s_{\min} = 10$ and $\kappa_r = 0$, so these dataset-invariant settings are reported once in Table 10 and omitted here. The "Ord." column follows the template priority order fixed by the selected deployment policy.

Dataset	Method	MCAR	Attr.-skewed	Tuple-block	Value-dep.
WANDS	Drop	0.803	0.803	0.814	0.802
	MeanMode	0.826	0.837	0.814	0.830
	MICE	0.825	0.832	0.814	0.830
	MissForest	0.849	0.821	0.805	0.834
	HoloClean	0.837	0.833	0.814	0.854
	Raha + Baran	0.855	0.879	<u>0.835</u>	0.842
	QueryCls	<u>0.863</u>	<u>0.882</u>	0.830	<u>0.858</u>
	MIFA	**0.912**	**0.930**	**0.867**	**0.907**
AppReviews	Drop	0.793	0.793	0.806	0.802
	MeanMode	0.942	0.942	0.899	0.882
	MICE	0.935	0.934	0.891	0.943
	MissForest	0.935	0.936	0.882	0.941
	HoloClean	0.793	0.793	0.806	0.802
	Raha + Baran	0.935	0.924	0.874	0.954
	QueryCls	<u>0.958</u>	<u>0.956</u>	**0.929**	**0.964**
	MIFA	**0.961**	**0.959**	<u>0.929</u>	<u>0.961</u>
Amazon − multi	Drop	0.838	0.838	<u>0.756</u>	0.819
	MeanMode	0.570	0.565	0.431	0.522
	MICE	0.851	0.851	<u>0.756</u>	0.819
	MissForest	**0.899**	<u>0.924</u>	0.754	0.850
	HoloClean	0.881	0.914	<u>0.756</u>	0.848
	Raha + Baran	0.838	**0.933**	0.514	<u>0.878</u>
	QueryCls	0.882	0.888	**0.789**	0.857
	MIFA	<u>0.906</u>	0.870	<u>0.756</u>	**0.903**

Table 12: F_1@10 under alternative missingness mechanisms. Best and second-best results within each dataset and mechanism are bolded and underlined, respectively.

tal budget unequally across the designated attributes according to workload frequencies estimated from the training workload. Tuple-block masks set $w > 1$ and jointly hide multiple query-relevant attributes in each selected tuple. For value-dependent missingness, the evaluator uses the complete pre-mask value only to sample whether its cell is hidden; the hidden value is never provided to any evaluated method. The applicable primary baselines participate under the same masks. We report F_1@k, precision@k, recall@k, false-admission and false-rejection rates, abstention, support coverage, affected-query rate, and masked-answer-slot rate.

Table 12 evaluates alternative missingness structures on mixed, numerical, and categorical workloads under the same overall incomplete-tuple budget. On WANDS, MIFA leads the strongest competing method by 3.2–4.9 F_1@10 points across all four mechanisms; on AppReviews, it remains within 0.3 points of the best result throughout. The harder case is tuple-block missingness on Amazon − multi, where MIFA drops from 0.906 under MCAR to 0.756 as multiple query-relevant attributes of the same tuple become simultaneously unobserved, whereas under value-dependent missingness it reaches 0.903 and exceeds the next best method by 2.5 points. This contrast is consistent with MIFA's support-backed joint eligibility inference: skewing which tuples or values are missing still leaves observed scalar and hierarchical context for retrieving compatible historical support, while removing several predicate attributes together makes such support sparser and forces more coarse-grained backoff or abstention. Overall, MIFA remains competitive across alternative missingness patterns, while tuple-block missingness exposes the principal stress point of joint eligibility inference.

Oracle answer-targeted stress. We additionally evaluate the controlled oracle stress described in Section 5.1. For each test query q, let

$$r_q = \text{round}\big(\gamma_{\text{ans}} |\text{Ans}^*(q, D^*_{\text{test}})|\big)$$

be its target number of exposed complete-data answer slots. Using a seeded query-wise round-robin procedure, the masker selects

answer tuples whose query-relevant designated attributes remain available for masking, subject to the same global budget B_{test}, per-attribute budgets, and $w = 1$ constraint as the non-oracle settings. If both designated attributes are referenced by the query, the masker uses the attribute with the larger remaining budget, with seeded tie-breaking. Once the requested answer exposure is reached or no further eligible answer cell can be masked under these constraints, the remaining global budget is filled uniformly from unselected tuples.

Because one masked tuple can belong to the top-k answers of multiple queries, the realized exposure need not equal the target γ_{ans}. We therefore report

$$\eta_{\text{ans}} = \frac{\#\{\text{Answer slots with a masked query-relevant cell}\}}{\#\{\text{complete-data answer slots}\}}.$$

We vary $\gamma_{\text{ans}} \in \{0.25, 0.5, 0.75, 1\}$ and report η_{ans}, the affected-query rate, answer quality, predicate error, abstention, and support coverage. These oracle results are reported separately and are never used for headline comparisons or model selection.

Table 13 progressively targets complete-data top-k answer slots while keeping the overall incomplete-tuple budget fixed, with η_{ans} measuring the stress actually realized by each workload. On WANDS, MIFA leads the strongest competitor by 26.2–27.8 F_1@10 points throughout the sweep; on AppReviews, realized exposure increases from 0.198 to 0.857 while MIFA decreases only from 0.945 to 0.920 and remains best at every operating point. This stability follows from retaining answer-critical tuples with missing predicates as unresolved and resolving their joint eligibility from compatible historical support: MIFA maintains support coverage of 1.000 with no inference-stage abstention in all 12 settings, whereas the saturation of η_{ans} at 0.245 on WANDS and 0.078 on Amazon − multi reflects the finite global and per-attribute masking budgets rather than a failure to resolve support. Consequently, MIFA remains effective when missingness is directed toward answer-critical predicate cells, while the Amazon − multi result at $\gamma_{\text{ans}} = 1$ also shows that realized exposure, rather than the requested γ_{ans} alone, is necessary for interpreting the severity of this stress.

Predicate selectivity. For a test query q, we define its complete-data predicate selectivity as

$$\text{sel}(q) = \frac{|E^*(q, D^*_{\text{test}})|}{|D^*_{\text{test}}|}.$$

For constructed predicates, constants are generated using training quantiles or frequencies; benchmark-provided filter constants are retained unchanged. After the queries have been fixed, the evaluator partitions them into the predeclared low-, medium-, and high-selectivity strata according to $\text{sel}(q)$. The complete test values are used only for this diagnostic stratification and ground-truth computation; they are not exposed to any method or used to select an operating point.

All methods receive the same queries within each stratum. The applicable primary baselines use the same fixed masks and validation-selected configurations. We report F_1@k, false-admission and false-rejection rates, abstention, support coverage, selected context levels, and the number of statistical resolutions. This analysis isolates the effect of predicate selectivity from predicate type and conjunction arity.

Tables 14 and 15 stratify the original main-workload queries by complete-data predicate selectivity and show that MIFA's relative gain changes across the three strata: MIFA leads throughout WANDS, leads in the low- and medium-selectivity strata of AppReviews, and gains mainly in the low-selectivity stratum of Amazon − multi. This behavior follows MIFA's execution path: observed predicate atoms and deterministic constraints first produce hard pass/fail decisions, while only distance-competitive unresolved candidates enter joint eligibility inference, so changing selectivity changes both the number and the composition of candidates for which historical support can affect eligibility. Accordingly, Res./q remains substantial across all three WANDS strata, decreases from 6.89 to 2.74–2.93 on AppReviews, and drops from 7.12 to 0.04 on Amazon − multi; with support coverage fixed at 1.000 and abstention at 0.000, these differences directly reflect how much statistical inference is exercised by each selectivity stratum. Predicate selectivity therefore affects end-to-end effectiveness through the unresolved-candidate set encountered during distance-ordered execution, which explains why the benefit of joint eligibility inference varies across strata while remaining consistent with the hard-decision and support-backed inference design of MIFA.

Historical-evidence size. We evaluate MIFA using nested subsets containing 25%, 50%, 75%, and 100% of D_{tr}. For entity-based workloads, the subsets are formed by nested entity groups; for timestamped workloads, they are chronological prefixes. To isolate tuple-side support evidence, the query workload, validation/calibration/test partitions, and controlled masks are held fixed across evidence sizes; in particular, Q_{hist} is not resampled in this diagnostic.

At each evidence size, we repeat candidate discovery, joint design selection, policy refitting, structure construction, and threshold calibration using only the corresponding training subset and the unchanged validation and calibration partitions. No tuple excluded from the selected training subset can contribute support evidence. We report support size, retained templates, selected context levels, backoff depth, abstention, predicate precision(P) and recall(R), false-admission(FA) and false-rejection(FR) rates, and F_1@k. This is a MIFA-only analysis because it measures the sensitivity of the proposed inference mechanism to the amount of historical support. We report representative results on WANDS and Amazon − multi, covering mixed categorical–numerical and categorical predicates.

Across the four evidence sizes, support coverage remains 1.0 and abstention remains zero on both workloads, while the retained template sets remain fixed at three templates on WANDS and two on Amazon − multi. WANDS remains stable across the sweep, with F_1@10 between 0.908 and 0.914 and a false-admission rate below 0.001 at every evidence size.

On Amazon − multi, F_1@10 reaches 0.917 with 50% historical evidence and remains unchanged through 100%, while using only 25% evidence reduces F_1@10 by about 0.010 to 0.907. All four evidence sizes require no hierarchical backoff, while the core shared support-index footprint grows approximately linearly from 0.31 to 1.24 MiB. Thus, answer quality saturates after 50% historical evidence, while additional evidence mainly increases core support-index storage.

Eligibility threshold. Keeping $\Gamma^\star$, the deployed structures $\mathcal{J}$, and

Dataset	γ_{ans}	η_{ans}	Drop	MeanMode	MICE	MissForest	HoloClean	Raha + Baran	QueryCls	MIFA
WANDS	0.25	0.169	0.130	0.233	0.233	0.400	0.520	0.148	0.560	**0.829**
	0.50	0.232	0.017	0.158	0.158	0.281	0.405	0.402	0.510	**0.789**
	0.75	0.244	0.002	0.145	0.145	0.276	0.421	0.357	0.560	**0.832**
	1.00	0.245	0.000	0.168	0.168	0.299	0.413	0.546	0.585	**0.847**
AppReviews	0.25	0.198	0.584	0.902	0.895	0.893	0.584	0.887	0.941	**0.945**
	0.50	0.483	0.326	0.859	0.844	0.837	0.326	0.845	0.929	**0.938**
	0.75	0.732	0.109	0.812	0.796	0.797	0.109	0.781	0.927	**0.928**
	1.00	0.857	0.000	0.780	0.748	0.738	0.000	0.736	0.917	**0.920**
Amazon − multi	0.25	0.073	0.010	0.233	0.034	0.321	0.046	0.401	0.501	**0.541**
	0.50	0.078	0.000	0.190	0.017	0.356	0.028	0.324	0.540	**0.601**
	0.75	0.078	0.000	0.179	0.024	0.396	0.005	0.353	0.425	**0.444**
	1.00	0.078	0.000	0.181	0.000	0.414	0.031	0.461	**0.561**	0.520

Table 13: Oracle answer-targeted stress under fixed incomplete-tuple budgets. η_{ans} is the realized answer-slot exposure; remaining entries report F_1@10.

Dataset	Sel.	n	Drop	MeanMode	MICE	MissForest	HoloClean	Raha+Baran	QueryCls	MIFA
WANDS	Low	80	0.793	0.814	0.810	0.815	0.796	0.740	0.819	**0.859**
	Medium	80	0.793	0.799	0.788	0.845	0.821	0.741	0.893	**0.964**
	High	80	0.774	0.841	0.840	0.838	0.745	0.838	0.843	**0.908**
AppReviews	Low	200	0.760	0.857	0.850	0.864	0.763	0.826	0.891	**0.902**
	Medium	200	0.666	0.978	0.971	0.967	0.687	0.918	0.981	**0.982**
	High	200	0.729	0.967	0.960	0.957	0.723	0.889	**0.981**	0.976
Amazon − multi	Low	100	0.836	0.783	0.836	0.846	0.804	0.826	0.798	**0.856**
	Medium	100	0.829	0.568	0.829	0.905	0.907	0.829	**0.913**	0.909
	High	100	0.885	0.247	0.885	0.921	0.945	0.885	**0.957**	0.889

Table 14: F_1@10 across predicate-selectivity strata; n is the number of queries in each stratum. Bold and underlined values indicate the best and second-best results in each row, respectively.

Dataset	Hist.	$\lvert\mathcal{I}_{\text{mv}}\rvert$ (MiB)	No backoff (%)	Depth	P	R	FA	FR	F_1@10
WANDS	25%	0.52	92.9	2.92	.943	.971	0.00072	0.0293	0.914
	50%	1.04	86.6	4.92	.940	.967	0.00076	0.0331	0.908
	75%	1.55	86.7	5.09	.950	.966	0.00062	0.0339	0.913
	100%	2.07	87.6	5.00	.926	.980	0.00096	0.0204	0.910
Amazon − multi	25%	0.31	100.0	0.00	0.992	0.934	0.00007	0.0658	0.907
	50%	0.62	100.0	0.00	0.997	0.934	0.00003	0.0658	0.917
	75%	0.93	100.0	0.00	0.997	0.934	0.00003	0.0658	0.917
	100%	1.24	100.0	0.00	0.997	0.934	0.00003	0.0658	0.917

Table 16: Historical-evidence sensitivity. P and R denote predicate precision and recall; FA and FR denote false-admission and false-rejection rates. No backoff is the share of statistical resolutions completed without hierarchical backoff, and Depth is the mean backoff depth when invoked.

Dataset	Sel.	FA	FR	Res./q	F_1@10
WANDS	Low	0.00076	0.0746	21.36	0.859
	Medium	0.00076	0.0028	16.11	0.964
	High	0.00138	0.0201	30.80	0.908
AppReviews	Low	0.00670	0.0677	6.89	0.902
	Medium	0.03267	0.0123	2.74	0.982
	High	0.03884	0.0081	2.93	0.976
Amazon − multi	Low	0.00009	0.1148	7.12	0.856
	Medium	0.00069	0.0535	4.04	0.909
	High	0.00133	0.0366	0.04	0.889

Table 15: MIFA diagnostics on the same selectivity strata. FA and FR denote false-admission and false-rejection rates, and Res./q is the number of statistical resolutions per query.

$\pi^\star$ fixed, we evaluate every threshold in the predeclared set

$$\Theta = \{0, 0.01, \ldots, 0.99, 1\}.$$

We report the answer-level F_1@10 sweep and use predicate precision, recall, false-admission, and false-rejection rates to interpret threshold sensitivity.

Let $\theta_{\text{cal}} = \theta^\star$ denote the operating threshold selected on D_{cal}. For diagnosis only, we also report the test-optimal threshold $\theta^\star_{\text{test}}$, the threshold gap $|\Delta\theta|$, and the resulting answer-quality selection regret ΔF_1. All reported operating results use θ_{cal}; $\theta^\star_{\text{test}}$ is never used for model selection or deployment. This is a MIFA-only analysis.

Figure 8 and Table 17 compare θ_{cal} with $\theta^\star_{\text{test}}$ while keeping $\Gamma^\star$, $\mathcal{J}$, and $\pi^\star$ fixed. The corresponding F_1@10 regrets are 0.001 on AppReviews, 0.014 on WANDS, and 0.000 on Amazon − multi. On WANDS, increasing θ from 0.40 to 0.53 raises predicate precision from 0.926 to 0.947 and reduces false admission from 0.00096 to 0.00066, while recall decreases from 0.980 to 0.968. On Amazon − multi, F_1@10 remains 0.917 across all 78 thresholds from $\theta = 0.23$ to 1.00, so $\theta_{\text{cal}} = 0.66$ and $\theta^\star_{\text{test}} = 1.00$ differ by 0.34 without answer-quality regret.

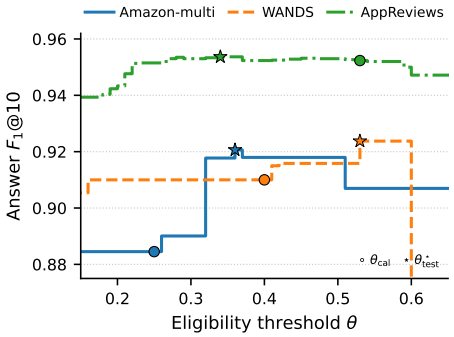


Figure 8: Eligibility-threshold sensitivity on WANDS, AppReviews, **and** Amazon – multi**. Circles mark θ_{cal} selected on D_{cal}; stars mark the diagnostic test-optimal $\theta^{\star}_{test}$.**

Dataset	θ_{cal}	$\theta^{\star}_{test}$	$\lvert\Delta\theta\rvert$	F_1^{cal}	$F_1^{test\star}$	ΔF_1
WANDS	0.40	0.53	0.13	0.910	0.924	0.014
AppReviews	0.53	0.34	0.19	0.952	0.954	0.001
Amazon – multi	0.66	1.00	0.34	0.917	0.917	0.000

Table 17: Calibration-selected and diagnostic test-optimal thresholds under the same fixed structure and policy. Here, $\lvert\Delta\theta\rvert = \lvert\theta^{\star}_{test} - \theta_{cal}\rvert$ and $\Delta F_1 = F_1@10(\theta^{\star}_{test}) - F_1@10(\theta_{cal})$.

Thus, threshold sensitivity depends on the resulting predicate precision–recall balance: AppReviews is nearly unchanged around θ_{cal}, whereas Amazon – multi exhibits a broad answer-quality plateau. These results support selecting $\theta^{\star}$ on D_{cal} only after $\Gamma^{\star}$, $\mathcal{J}$, and $\pi^{\star}$ are fixed; ΔF_1 quantifies the remaining sensitivity of this calibration step.

D.3 Additional Efficiency and Scalability Results

Efficiency under varying workloads. Using the Exp-1 runs, we evaluated MIFA's P95 CPU latency while varying missingness exposure β_{exp}, predicate type and arity, and k.

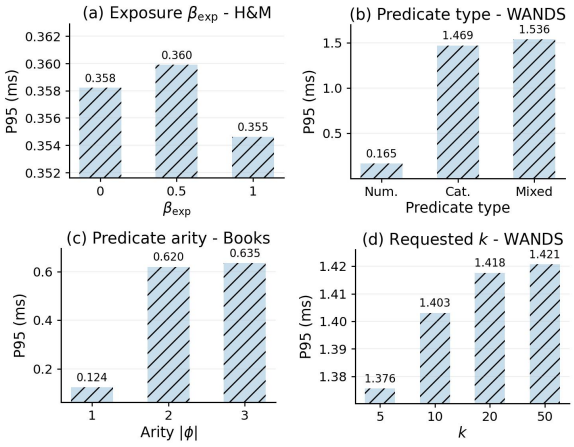


Figure 9: MIFA**'s P95 CPU latency under varying missingness exposure, predicate type and arity, and result size k.**

As shown in Figure 9, latency is more sensitive to predicate structure than to missingness placement or result-set size. Varying β_{exp} changes H&M latency only within 0.355–0.360 ms, and increasing k from 5 to 50 raises WANDS latency only from 1.376 to 1.421 ms. In contrast, latency rises from 0.165 to 1.536 ms from numerical to mixed predicates and from 0.124 to 0.635 ms as arity increases from one to three. Missingness placement does not widen the predicate, and larger k mainly affects final selection, whereas mixed and multi-atom predicates require more joint eligibility processing. Thus, MIFA remains efficient across missingness placements and result sizes, with additional cost concentrated in harder predicates.

All-dataset efficiency. We report MIFA on the nine standard HCE workloads and two ArXiv – large corpus scales using the CPU, one-GPU, and two-GPU serving paths. The one-GPU setting uses one serving process on one device; the two-GPU setting uses two concurrent serving replicas, with aggregate throughput and the slower-replica latency reported.

Table 18 reports a separate platform sweep over the nine standard workloads and two ArXiv – large scales, with workload-specific F_1@10 unchanged across execution platforms. On the nine standard workloads, the one-GPU path reaches 6.46K–57.42K QPS and two concurrent GPU replicas reach 12.82K–117.03K QPS, while P95 latency remains below 1.8 ms on all nine. The two-GPU throughput is close to twice the one-GPU throughput across these workloads, reflecting replica-level parallel serving rather than a change in query semantics or eligibility inference. This confirms that the same MIFA serving version can retain its answer quality while scaling throughput through independent GPU replicas, complementing the CPU-focused comparison and corpus-size scaling reported in the main experiments.

Construction time and memory. We compare MIFA's offline construction with FAVOR and SIEVE paired with Drop, MeanMode, MICE, MissForest, HoloClean, and Raha + Baran on representative mixed, numerical, and categorical workloads. All methods use the same data snapshot, embeddings, scalar representation, hardware, and number of worker threads. Backend parameters are selected using the same validation search-quality targets as in the main experiments.

For MIFA, we separately report the time and persistent footprint of the plug-in ANN index $\mathcal{I}_e$, projected partition tree $\mathcal{T}_e$, shared support index $\mathcal{I}_{mv}$, and deployment policy $\pi^{\star}$. For $\mathcal{T}_e$, we separately attribute search metadata (*e.g.,* leaf lists and projected MBRs) and context-routing metadata where possible, while counting their shared topology only once. We additionally report policy discovery, validation-based joint selection, final refitting, and threshold-calibration time separately from serving-structure construction.

For each static baseline, we report wrapper fitting and materialization time separately from backend-index construction. For MeanMode, fitting includes estimating the training statistics, while materialization includes applying the selected values to the searchable snapshot. For MICE, MissForest, HoloClean, and Raha + Baran, we likewise report fitting or repair time and one-time materialization separately from the filtered-search backend. The end-to-end preparation time is the sum of these components.

Memory measurements report both peak resident memory during construction and the ready-to-query persistent footprint beyond the raw tuple and embedding arrays shared by all methods. For

Dataset	$\|D\|$	d	Platform	F_1@10	P50 (ms)	P95 (ms)	QPS	Peak RSS (GiB)
WANDS	6,349	1,024	CPU	0.910	1.371	1.443	727.4	2.04
			1-GPU	0.910	1.143	1.389	6,460.3	2.02
			2-GPU	0.910	1.172	1.395	12,816.3	4.04
Books	1,600	1,024	CPU	0.895	0.604	0.640	1,659.7	1.58
			1-GPU	0.895	1.084	1.273	7,285.7	1.46
			2-GPU	0.895	1.070	1.254	14,634.8	2.92
MovieLens	2,642	1,024	CPU	0.934	0.113	0.139	8,619.6	1.64
			1-GPU	0.934	0.665	0.799	9,863.4	1.51
			2-GPU	0.934	0.682	0.898	21,429.7	3.02
Yelp	11,890	1,024	CPU	0.954	1.265	2.125	884.0	2.26
			1-GPU	0.954	1.099	1.214	8,607.7	2.45
			2-GPU	0.954	1.079	1.232	17,370.7	4.91
H&M	21,025	1,024	CPU	0.970	0.157	0.422	5,213.8	3.32
			1-GPU	0.970	1.283	1.616	11,350.5	3.56
			2-GPU	0.970	1.265	1.598	22,811.3	7.12
Amazon – multi	5,359	1,024	CPU	0.917	1.169	1.209	855.8	1.89
			1-GPU	0.917	1.084	1.325	8,570.8	1.84
			2-GPU	0.917	1.093	1.184	17,113.2	3.67
AppReviews	57,613	1,024	CPU	0.953	0.806	1.259	1,231.5	7.14
			1-GPU	0.953	1.485	1.639	8,364.5	7.09
			2-GPU	0.953	1.472	1.615	16,707.1	14.23
Amazon – C4	12,000	1,024	CPU	0.768	0.420	1.754	1,307.1	2.30
			1-GPU	0.768	0.694	0.825	57,418.1	2.47
			2-GPU	0.768	0.720	0.858	117,030.3	4.92
ArXiv	20,000	4,096	CPU	0.742	1.201	2.060	776.1	9.06
			1-GPU	0.742	1.539	1.776	8,480.6	9.00
			2-GPU	0.742	1.541	1.794	17,036.0	18.01
ArXiv – large – 1M	1,000,000	128	CPU	0.799	0.935	1.090	1,108.6	8.90
			1-GPU	0.799	1.272	1.605	3,781.0	8.70
			2-GPU	0.799	1.242	1.593	6,195.6	17.40
ArXiv – large – 2.73M	2,735,264	128	CPU	0.817	1.717	2.011	583.2	11.57
			1-GPU	0.817	1.521	1.903	3,055.6	12.02
			2-GPU	0.817	1.531	1.958	5,926.9	24.03

Table 18: MIFA **efficiency on nine standard HCE workloads and two** ArXiv – large **scales using one or two RTX 4090 GPUs.**

MIFA, we also report the fraction of persistent space consumed by each serving structure. No offline construction or materialization time is included in the online query latency reported in Exp-2; its amortized contribution is reported separately under the fixed number of queries per construction or refresh.

Tables 21–23 separate handler preparation, index construction, and MIFA's offline design on mixed, numerical, and categorical workloads. For the static pipelines, backend construction remains a small part of end-to-end preparation once repair becomes substantial: for example, HoloClean contributes 31.79–58.56 s of repair time on AppReviews and WANDS, while Raha + Baran contributes 223.41–1401.51 s across the three workloads. MIFA spends 41.74, 37.47, and 174.50 s in total on WANDS, AppReviews, and H&M, respectively, and its phase breakdown shows that this cost is concentrated in validation-based joint selection and final refitting rather than in serving-structure construction, which takes 4.58–8.68 s. The persistent-state breakdown follows the structures selected for each workload: $\mathcal{I}_{\text{mv}}$ accounts for 90.79% and 71.17% of MIFA's state on WANDS and H&M, whereas $\mathcal{T}_e$ search metadata accounts for 80.17% on AppReviews; the deployment policy remains below 0.1 MiB in all three cases. Thus, MIFA places its main construction cost in offline structure–policy selection and refitting, while the deployed state consists primarily of the tree and shared support structures used by distance-ordered search and joint eligibility inference. With 10,000 queries per construction or refresh, the measured offline preparation corresponds to 4.17, 3.75, and 17.45 ms per query on the three workloads, respectively.

Additional scalability dimensions. In addition to varying $|D|$ in the main experiments, we vary the embedding dimension d and the number of query-relevant scalar attributes $|\bar{\mathcal{A}}|$, changing one dimension at a time.

Varying embedding dimension. We evaluate embedding dimension scaling with the Matryoshka representations of Qwen3 – Embedding – 0.6B on H&M (categorical), WANDS (mixed), and AppReviews (numerical). At each width, the truncated representation is used only for candidate generation, while the same 1024-dimensional exact reranking path is kept unchanged. Figure 10 shows a clean quality–resource trade-off across all three predicate regimes. At 512 dimensions, all three workloads preserve the 1024-dimensional F_1@10 exactly while halving the candidate vector index. At 256 dimensions, the equal-weight mean F_1@10 remains 0.9936, the candidate vector index is reduced by 75%, CPU P95 improves by a geometric-mean 2.20×, and 1GPU P95 by 1.59× relative to 1024 dimensions. The more aggressive 128-dimensional point retains mean F_1@10 = 0.9851, removes 87.5% of candidate-index bytes, and further improves CPU/1GPU P95 by 2.44×/1.66×. Shorter Matryoshka prefixes therefore reduce candidate-state footprint and the memory traffic and arithmetic of candidate generation, while preserving the same exact reranking semantics. MRL is complementary to MIFA: MRL reduces the storage and unit cost of vector retrieval, whereas MIFA's pruning

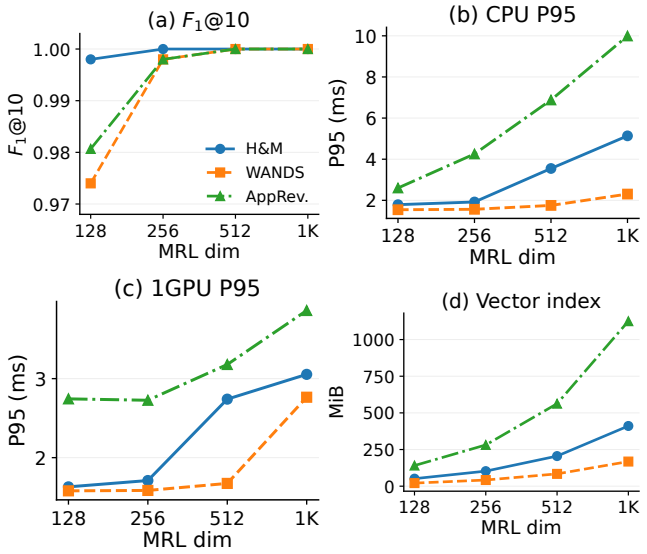


Figure 10: Matryoshka-representation quality, retrieval cost, and vector-index footprint.

and deferred eligibility inference reduce the amount of search and inference work performed per query.

Varying scalar-schema size. For the scalar-attribute experiment, we use nested attribute sets fixed before test execution. Predicate arity, incomplete-tuple rate, the default missingness mechanism, k, and the selectivity distribution are held fixed as closely as possible, so that the experiment measures the effect of a larger scalar schema rather than a different query workload. The designated missing-attribute set remains fixed whenever it is contained in the current scalar schema. Each method repeats its validation-only configuration selection at every scale; test results are not used to reuse or modify an operating point.

Every applicable static filtered-ANN backend participates with Drop and HoloClean. We report construction time, peak and persistent memory, median and P95 query latency, throughput, exact distance evaluations, statistical resolutions, and $F_1@k$. The quality metric ensures that lower construction or query cost is not obtained merely by using a lower-quality operating point. This scalar-schema-only ablation uses the controlled MovieLens and Yelp relations, containing 2,642 and 11,890 searchable tuples, respectively. We use P50, P95, and QPS here to measure changes across $|\bar{\mathcal{A}}|$ within these controlled workloads, while the main tables provide the cross-method end-to-end efficiency comparison.

Tables 19 and 20 vary $|\bar{\mathcal{A}}|$ from two to four while fixing the mixed predicate workload on MovieLens and Yelp. Across all methods, $F_1@10$ is unchanged across the three widths; within each method, the width-2 to width-4 P95 change is at most 0.020 ms on MovieLens and 0.085 ms on Yelp, and the exact-distance count remains unchanged. For MIFA, the selected structure, policy, and threshold also remain unchanged, with 25.92 statistical resolutions per query on MovieLens and 52.11 on Yelp at all three widths. The stable execution work follows from the fixed predicate and the unchanged serving configuration, while the additional scalar attributes are reflected in the maintained index state: MIFA's support-index component grows from 0.34 to 0.61 MiB on MovieLens and from 1.53 to 2.72 MiB on Yelp, with the candidate-provider and tree components unchanged; correspondingly, its total deployed state grows only from 12.37 to 12.64 MiB and from 55.15 to 56.34 MiB. The static indexes exhibit backend-specific state growth, most visibly for SIEVE, whose index grows from 73.17 to 83.99 MiB on MovieLens and from 54.36 to 128.05 MiB on Yelp. Thus, under a fixed mixed-predicate workload, increasing scalar-schema width primarily changes the amount of maintained scalar state rather than the geometric or eligibility work performed per query, matching MIFA's design in which additional scalar evidence is incorporated through the shared support index while the selected distance-ordered execution path remains fixed.

E SHARED SUPPORT-INDEX CONSTRUCTION AND RETRIEVAL

This appendix details the shared support index $\mathcal{I}_{\text{mv}}$ introduced in Section 4.2.2. It organizes observed scalar evidence from the training data across retained templates and hierarchical contexts without materializing a separate support list for every template–node pair.

Index construction. After fixing the partition tree $\mathcal{T}_e$, MIFA builds a single $\mathcal{I}_{\text{mv}}$ over D_{tr} for all retained templates $r : X_s, [\text{Loc}(A_e)] \rightsquigarrow Y$. It stores: (1) for each categorical body attribute, postings from observed values to training-tuple identifiers; (2) for each numerical body attribute, postings over buckets induced by the finest retained matching ranges; (3) for each retained target attribute, identifiers of training tuples in which it is observed; and (4) for each training tuple, the depth-first identifier of the leaf to which $\mathcal{T}_e$ routes it. Because depth-first numbering makes the leaves of each subtree contiguous, the last component represents any hierarchical context as a leaf interval. All components are shared across templates and contexts, and only D_{tr} contributes statistical support.

Support retrieval. Given a target tuple t and retained template $r : X_s, [\text{Loc}(A_e)] \rightsquigarrow Y$, MIFA instantiates the template's categorical keys and numerical ranges from $t[X_s]$. For each numerical condition, it retrieves the intersecting buckets and checks the exact condition to remove bucket-level false matches. It then intersects the body matches with the identifiers of training tuples whose target attributes Y are observed. If the template contains $\text{Loc}(A_e)$, the selected context node N further restricts the result to its contiguous leaf interval. The resulting tuples satisfy the template's scalar and locality conditions and have observed targets, providing exactly the support used by the hierarchical joint estimator in Section 4.2.3.

Dataset	$\lvert\mathcal{A}\rvert$	Method	Handler	F_1@10	P50 (ms)	P95 (ms)	QPS	Dist./q
MovieLens	2	MIFA	Native	**0.874**	0.745	0.796	1337.1	2642.0
	2	FAVOR	Drop	0.849	0.493	0.754	1769.0	387.3
	2	FAVOR	HoloClean	0.847	0.504	0.780	1724.6	388.6
	2	SIEVE	Drop	0.850	0.023	0.041	37802.9	42.0
	2	SIEVE	HoloClean	0.849	0.024	0.042	36018.1	42.6
	3	MIFA	Native	**0.874**	0.724	0.769	1374.9	2642.0
	3	FAVOR	Drop	0.849	0.498	0.774	1741.7	387.3
	3	FAVOR	HoloClean	0.847	0.505	0.772	1716.9	388.5
	3	SIEVE	Drop	0.850	0.024	0.044	36174.2	42.0
	3	SIEVE	HoloClean	0.849	0.023	0.043	36978.9	42.5
	4	MIFA	Native	**0.874**	0.725	0.778	1371.7	2642.0
	4	FAVOR	Drop	0.849	0.501	0.774	1741.4	387.3
	4	FAVOR	HoloClean	0.847	0.497	0.780	1734.2	388.6
	4	SIEVE	Drop	0.850	0.024	0.044	35869.4	42.0
	4	SIEVE	HoloClean	0.849	0.023	0.043	36825.6	42.6
Yelp	2	MIFA	Native	**0.786**	2.123	2.192	470.3	11890.0
	2	FAVOR	Drop	0.716	1.576	1.811	625.6	525.7
	2	FAVOR	HoloClean	0.716	1.596	1.862	613.7	525.7
	2	SIEVE	Drop	0.765	0.047	0.095	17962.1	163.3
	2	SIEVE	HoloClean	0.765	0.046	0.093	18557.2	163.3
	3	MIFA	Native	**0.786**	2.097	2.160	475.8	11890.0
	3	FAVOR	Drop	0.716	1.614	1.872	615.1	525.7
	3	FAVOR	HoloClean	0.716	1.617	1.892	604.8	525.7
	3	SIEVE	Drop	0.765	0.047	0.091	18274.7	163.3
	3	SIEVE	HoloClean	0.765	0.046	0.091	18576.3	163.3
	4	MIFA	Native	**0.786**	2.131	2.193	468.6	11890.0
	4	FAVOR	Drop	0.716	1.623	1.897	608.7	525.7
	4	FAVOR	HoloClean	0.716	1.603	1.871	611.3	525.7
	4	SIEVE	Drop	0.765	0.047	0.097	17872.5	163.3
	4	SIEVE	HoloClean	0.765	0.045	0.093	18359.9	163.3

Table 19: Scalar-schema-width quality and online execution on MovieLens **and** Yelp**.**

Dataset	$\lvert\mathcal{A}\rvert$	Method	Handler	Build (s)	Peak RSS (GiB)	Persistent (MiB)	Index (MiB)
MovieLens	2	MIFA	Native	1.022	1.53	12.37	12.37
	2	FAVOR	Drop	0.234	0.81	846.08	11.06
	2	FAVOR	HoloClean	0.242	0.81	845.96	11.06
	2	SIEVE	Drop	0.119	1.05	886.67	73.17
	2	SIEVE	HoloClean	0.121	1.05	886.50	72.81
	3	MIFA	Native	1.023	1.61	12.48	12.48
	3	FAVOR	Drop	0.241	0.81	846.11	11.06
	3	FAVOR	HoloClean	0.235	0.81	846.05	11.06
	3	SIEVE	Drop	0.143	1.05	890.71	77.22
	3	SIEVE	HoloClean	0.121	1.05	891.15	77.14
	4	MIFA	Native	1.014	1.69	12.64	12.64
	4	FAVOR	Drop	0.235	0.81	846.23	11.06
	4	FAVOR	HoloClean	0.239	0.81	846.37	11.06
	4	SIEVE	Drop	0.127	1.06	897.94	83.99
	4	SIEVE	HoloClean	0.106	1.06	897.31	83.61
Yelp	2	MIFA	Native	1.611	2.25	55.15	55.15
	2	FAVOR	Drop	1.124	1.40	1386.03	49.76
	2	FAVOR	HoloClean	1.183	1.40	1386.98	49.76
	2	SIEVE	Drop	0.220	1.41	1292.39	54.36
	2	SIEVE	HoloClean	0.244	1.41	1295.68	54.37
	3	MIFA	Native	1.642	2.58	55.63	55.63
	3	FAVOR	Drop	1.128	1.40	1387.69	49.76
	3	FAVOR	HoloClean	1.169	1.40	1387.84	49.76
	3	SIEVE	Drop	0.225	1.41	1298.26	54.49
	3	SIEVE	HoloClean	0.281	1.41	1297.59	55.36
	4	MIFA	Native	1.627	2.26	56.34	56.34
	4	FAVOR	Drop	1.138	1.40	1389.54	49.76
	4	FAVOR	HoloClean	1.161	1.40	1388.80	49.76
	4	SIEVE	Drop	0.268	1.51	1372.24	128.05
	4	SIEVE	HoloClean	0.277	1.51	1376.12	128.72

Table 20: Scalar-schema-width construction and memory on MovieLens **and** Yelp**.**

Dataset	Backend	Handler	Fit/Repair	Materialize (s)	Build	E2E Prep.	Amort.@10K (ms/q)	Peak RSS (GiB)	Persistent (MiB)
WANDS	FAVOR	Drop	0.000	0.000	0.635	0.635	0.063	1.07	26.57
	SIEVE	Drop	0.000	0.000	0.312	0.312	0.031	1.70	192.10
	FAVOR	MeanMode	0.005	0.004	0.624	0.633	0.063	1.08	28.08
	SIEVE	MeanMode	0.005	0.004	0.314	0.323	0.032	1.70	198.65
	FAVOR	MICE	0.605	0.040	0.637	1.283	0.128	1.06	28.10
	SIEVE	MICE	0.605	0.040	0.392	1.038	0.104	1.91	267.04
	FAVOR	MissForest	6.598	0.710	0.629	7.938	0.794	1.88	28.10
	SIEVE	MissForest	6.598	0.710	0.361	7.670	0.767	1.88	226.39
	FAVOR	HoloClean	58.562	0.028	0.633	59.223	5.922	2.08	27.16
	SIEVE	HoloClean	58.562	0.028	0.318	58.907	5.891	2.08	210.09
	FAVOR	Raha + Baran	223.411	0.062	0.653	224.127	22.413	10.15	27.17
	SIEVE	Raha + Baran	223.411	0.062	0.339	223.813	22.381	10.15	214.09
	MIFA	Native	0.000	0.000	5.052	41.742	4.174	1.66	50.41
AppReviews	FAVOR	Drop	0.000	0.000	19.698	19.698	1.970	6.63	241.10
	SIEVE	Drop	0.000	0.000	3.334	3.334	0.333	6.63	474.47
	FAVOR	MeanMode	0.009	0.006	19.586	19.601	1.960	6.63	246.35
	SIEVE	MeanMode	0.009	0.006	3.505	3.520	0.352	6.63	494.55
	FAVOR	MICE	0.720	0.085	7.957	8.762	0.876	4.39	243.67
	SIEVE	MICE	0.720	0.085	5.428	6.233	0.623	11.96	2318.14
	FAVOR	MissForest	5.886	0.558	8.526	14.970	1.497	4.39	243.67
	SIEVE	MissForest	5.886	0.558	5.350	11.794	1.179	11.56	2228.52
	FAVOR	HoloClean	31.786	0.080	19.388	51.255	5.125	6.63	243.67
	SIEVE	HoloClean	31.786	0.080	3.388	35.254	3.525	6.63	488.22
	FAVOR	Raha + Baran	906.778	0.259	7.718	914.755	91.475	14.44	243.67
	SIEVE	Raha + Baran	906.778	0.259	2.490	909.527	90.953	14.44	490.89
	MIFA	Native	0.000	0.000	8.678	37.470	3.747	6.71	60.10
H&M	FAVOR	Drop	0.000	0.000	2.266	2.266	0.227	2.86	87.99
	SIEVE	Drop	0.000	0.000	0.332	0.332	0.033	2.86	94.11
	FAVOR	MeanMode	0.014	0.014	2.257	2.285	0.229	2.86	96.25
	SIEVE	MeanMode	0.014	0.014	0.343	0.371	0.037	2.86	107.84
	FAVOR	MICE	0.803	0.163	2.203	3.169	0.317	2.02	91.39
	SIEVE	MICE	0.803	0.163	0.371	1.337	0.134	2.02	97.65
	FAVOR	MissForest	4.093	0.248	2.271	6.612	0.661	2.03	91.38
	SIEVE	MissForest	4.093	0.248	0.393	4.734	0.473	2.02	98.78
	FAVOR	HoloClean	42.893	0.073	2.228	45.195	4.519	2.86	90.40
	SIEVE	HoloClean	42.893	0.073	0.332	43.299	4.330	2.86	96.53
	FAVOR	Raha + Baran	1401.510	0.184	2.255	1403.949	140.395	43.43	90.39
	SIEVE	Raha + Baran	1401.510	0.184	0.344	1402.039	140.204	43.43	99.46
	MIFA	Native	0.000	0.000	4.578	174.504	17.450	3.12	53.02

Table 21: Offline construction and memory accounting on representative mixed, numerical, and categorical workloads. For MIFA**, Build reports serving-structure construction only, whereas E2E Prep. equals the full offline total in Table 23. Amort.@10K uses 10,000 queries per construction or refresh.**

Dataset	Serving component	Construction (s)	Persistent (MiB)	Share (%)
WANDS	$\mathcal{I}_e$	0.000	0.000	0.00
	$\mathcal{T}_e$ search	0.719	4.511	8.95
	$\mathcal{T}_e$ context	0.058	0.121	0.24
	$\mathcal{I}_{\text{mv}}$	0.023	45.773	90.79
	$\pi^\star$	4.247	0.009	0.02
AppReviews	$\mathcal{I}_e$	0.000	0.000	0.00
	$\mathcal{T}_e$ search	7.473	48.179	80.17
	$\mathcal{T}_e$ context	0.619	1.099	1.83
	$\mathcal{I}_{\text{mv}}$	0.067	10.757	17.90
	$\pi^\star$	0.486	0.064	0.11
H&M	$\mathcal{I}_e$	0.000	0.000	0.00
	$\mathcal{T}_e$ search	3.585	14.884	28.07
	$\mathcal{T}_e$ context	0.271	0.401	0.76
	$\mathcal{I}_{\text{mv}}$	0.087	37.737	71.17
	$\pi^\star$	0.622	0.002	0.00

Table 22: Construction time and persistent-space attribution of the MIFA **serving structures. The shared tree topology is counted once in the search row.**

Dataset	Discovery	Joint Sel.	Refit	Calibration (s)	Serving Build	Total
WANDS	0.807	17.691	17.240	0.953	5.052	41.742
AppReviews	0.253	10.091	9.326	9.123	8.678	37.470
H&M	1.334	75.773	75.004	17.815	4.578	174.504

Table 23: Offline phase breakdown of MIFA on the same three workloads.